Expert Systems With Applications 298 (2026) 129652

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Composite human activity recognition utilizing knowledge distillation and sensor fusion focusing on resource constrained microcontrollers

Athar Noor Mohammad Rafee [a,b,*], John Clear [c], Jannatun Noor [a,b]

[a] Computing for Sustainability and Social Good (C2SG) Research Group, Department of Computer Science and Engineering, United International University, Dhaka, Bangladesh
[b] Department of Computer Science and Engineering, School of Data and Sciences, BRAC University, Dhaka, Bangladesh
[c] Department of Computer Engineering, Charles W. Davidson College of Engineering, San José State University, San José, CA, United States of America

ARTICLE INFO

Keywords:
TinyML
Low-cost
Composite human activity recognition
Microcontrollers
Resource constrained AI
Nural networks
Knowledge distillation

ABSTRACT

This study presents a cost-effective, low-computation system for composite Human Activity Recognition (HAR) that leverages knowledge-distilled neural networks on a Microcontroller Unit (MCU) to minimize reliance on cloud processing. A key contribution of this work is the investigation of plantar pressure sensor data within a knowledge distillation framework, addressing a notable gap in the existing literature. The proposed solution centers around the ESP32-S3 DevKit C1, equipped with a dual-core 240 MHz Tensilica chip, 320 KiB of usable Static Random Access Memory (SRAM), and built-in Wi-Fi and Bluetooth. Significantly, both the teacher and the student models surpass existing state-of-the-art methods, achieving F1-scores of 99.33 %, 98.36 %, and 97.68 % respectively, in classifying a comprehensive set of 21 activities (15 composite and 6 simple). The distilled student models demonstrate remarkable efficiency, with execution times of 1.83 and 0.64 s, memory footprints of only 62 KB and 82 KB, and flash memory usage of approximately 209 KB and 127 KB, while maintaining low power consumption of 210 mW and 215 mW, respectively. Furthermore, we have developed an end-to-end prototype that integrates the ESP32-S3 with a WitMotion Inertial Measurement Unit (IMU) sensor. This system autonomously manages data acquisition, feature extraction, and inference in under 7 s with a total power consumption of approximately 295 mW.

1. Introduction

Human activity recognition has gained significant attention in recent years due to its wide range of applications in areas such as health monitoring, fitness tracking, and smart home automation. Recognizing activities such as walking, sitting, or running enables systems to offer personalized insights and human-centric automation that improves user experience and safety (Kaur et al., 2024; Khan et al., 2025; sedaghati et al., 2025). However, as the complexity of human activities increases, especially in composite activity recognition (Chen & Nugent, 2019) where multiple activities are performed concurrently or sequentially, the demand for computational resources also rises.

1.1. Motivation

Traditional Human Activity Recognition (HAR) systems often rely on cloud-based processing, where sensor data is transmitted to remote servers for analysis. While this approach leverages powerful computing infrastructure, it introduces significant drawbacks. These issues are particularly amplified for composite HAR. For instance, the continuous data streaming required to recognize longer, multi-stage activities inherently increases latency and energy consumption, which can undermine the effectiveness of real-time applications. Similarly, the detailed personal routines revealed by composite activity data create heightened privacy risks when transmitted and stored externally (Cao et al., 2024; Murshed et al., 2021). Beyond these concerns, managing such a large-scale distributed cloud-based solution is increasingly complicated, costlier, and requires expertise. As a result, on-device end-to-end Tiny Machine Learning (TinyML) solution is getting attention like never before (Elhanashi et al., 2024; Tsoukas et al., 2024).

The challenge lies in performing composite human activity recognition on resource-constrained devices, such as microcontrollers, where computational power, memory, and energy are severely limited (Qi et al., 2024). With the advent of Internet of Things (IoT), there is a growing need for HAR systems that can process data locally on devices to reduce dependency on cloud servers. On-device processing not only

* Corresponding author.
E-mail addresses: noor.mohammad.rafee@g.bracu.ac.bd (A.N.M. Rafee), john.clearIv@sjsu.edu (J. Clear), jannatun.noor@bracu.ac.bd (J. Noor).

https://doi.org/10.1016/j.eswa.2025.129652
Received 8 July 2025; Received in revised form 7 August 2025; Accepted 5 September 2025
Available online 10 September 2025
0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

reduces latency and network strain but also addresses privacy concerns by minimizing the need to transmit personal data (Wang et al., 2025b). However, implementing effective Deep Learning (DL) algorithms on low-power, low-cost Microcontroller Units (MCUs) remains a significant challenge due to hardware limitations. Many state-of-the-art HAR models are computationally intensive, involving complex architectures that are not feasible for MCUs, forcing the adoption of simpler architecture at the cost of accuracy.

1.2. Limitations

Current approaches to solving this problem largely rely on cloud or hybrid models, where raw sensor data is captured on the device and transmitted to a remote server for processing and analysis. Although this offloads the computation from the device to more powerful infrastructure, it introduces latency and requires constant connectivity, which is not ideal for many real-world scenarios. For applications like health monitoring and fitness tracking, where near-real-time feedback is crucial, relying on cloud processing may result in unacceptable delays. Additionally, the energy cost of transmitting data, especially in battery-operated devices, can significantly reduce operational time, making these systems impractical for long-term use in portable applications. Efforts to move HAR onto MCUs have mostly involved basic Machine Learning (ML) models with limited complexity (Daghero et al., 2022a; Tsoukas et al., 2024), often sacrificing accuracy or the number of classifiable labels to meet hardware constraints. These trade-offs between accuracy, detection capability, and resource usage present a significant barrier to widespread adoption of HAR related tasks.

To address these limitations, recent research has increasingly focused on Knowledge Distillation (KD) (Kagiyama & Okita, 2025; Moslemi et al., 2024) as a potent model compression technique. KD facilitates the transfer of learned representations, often referred to as “dark knowledge,” from a large, complex Neural Network (NN) (the teacher) to a smaller, more computationally efficient NN (the student). This process enables the deployment of the compact student model on resource-constrained edge devices such as MCUs, single-board computers, and mobile phones, without a substantial loss in predictive performance. A particularly salient application of KD within the HAR domain is multi-sensor fusion (Muniasamy & Asiri, 2025). In this paradigm, a sophisticated teacher model is trained on a rich dataset comprising inputs from multiple sensors. Subsequently, a student model is trained on a subset of this data, frequently from only a single sensor. This methodology significantly curtails the student model’s complexity while allowing it to retain a significant portion of the teacher’s comprehensive understanding, which was distilled from the multi-modal sensory inputs (Moslemi et al., 2024). When the sensors involved capture information from different modalities (e.g., motion and physiological signals), this approach is more specifically termed cross-modal KD.

While existing literature has explored these KD techniques to some extent using conventional inertial sensors like Inertial Measurement Units (IMUs), which include accelerometers, gyroscopes, and magnetometers, a notable and highly informative modality remains largely unexplored in this context. Specifically, the Plantar Pressure (PP) sensor (De Guzman et al., 2022; Li et al., 2018), which captures dynamic pressure distribution across the foot-ground interface (Liu et al., 2021), offers a rich source of data for analyzing gait, balance, and specific foot-related movements (Burnie et al., 2024; Truong et al., 2021). Unlike IMUs that measure kinematic motion, PP sensors provide kinetic data (Polo-Rodríguez et al., 2025) that can reveal subtle nuances in activities such as walking, running, or standing up. Despite its potential to significantly enhance activity recognition accuracy and provide deeper biomechanical insights, this sensor modality has been conspicuously overlooked within the domain of HAR. The application of PP sensors in the broader HAR domain is itself nascent, with only a limited number of studies (Li et al., 2025; Luo et al., 2024; Zhu et al., 2024) investigating their potential, and often in a limited capacity.

Consequently, the application of KD-based methodologies, particularly cross-modal distillation, to PP sensor data represents a significant and unaddressed research gap. Furthermore, there is a distinct lack of comprehensive analysis employing both conventional ML and advanced DL models on this data modality. This gap underscores a compelling opportunity for novel research that could substantially advance the state-of-the-art in wearable HAR systems.

1.3. Our contribution

As a result, in this study, we propose a prediction platform to composite human activity recognition utilizing PP that is optimized for deployment on resource-constrained microcontroller platforms. Our key contributions are:

I. We design DualCrossTAP, a custom two-branch neural network that achieves 99.33 % accuracy, surpassing existing state-of-the-art models in CAPPIMU dataset. Next, we propose two resource-efficient Convolutional Neural Network (CNN) models tailored for low-compute MCUs, balancing high performance with hardware constraints, while maintaining deployability on edge devices with stringent hardware constraints.
II. We adapt a standard KD method that integrates PP and IMU sensors, effectively transferring dark knowledge from the DualCrossTAP model to the student CNN models. The student models achieve up to 98.35 % accuracy and up to a 23× compression ratio, outperforming all existing state-of-the-art models.
III. To the best of our knowledge, this work is the first to systematically investigate the role of PP sensors within a KD-based sensor fusion context. We uniquely explore bidirectional knowledge transfer, examining distillation from PP to IMU as well as from IMU to PP, thereby addressing a significant research gap in the HAR domain.
IV. Subsequently, we perform a comprehensive parameter and Million Floating Point Operations (MFLOPs) analysis on the recent state-of-the-art models, demonstrating our models’ superior efficiency-performance balance. We then deploy the student CNN models on an ESP32-S3 MCU, showcasing its real-world viability for low-cost, low-power, and resource-constrained applications.
V. Finally, we develop a complete end-to-end prototype system for composite HAR, integrating the ESP32-S3 with a WitMotion IMU sensor. This prototype autonomously handles raw sensor data acquisition, on-device preprocessing, feature extraction, and inference, delivering a prediction in under seven seconds while consuming only 295 mW of power.

1.4. Organization of this study

The rest of the paper is structured as follows: Section 2 reviews the relevant literature and compares recent advancement in the HAR domain with respect to TinyML, sensor utilization, practical deployability, and feasibility. Section 3 provides a detailed overview of the CAPPIMU dataset, including data preparation, preprocessing pipelines, and the rationale for the chosen train-test splitting strategies. In Section 4, we delineate the architecture of our proposed models, discuss the rationale behind key design decisions, and describe the employed KD approach, the selection of the MCU, and the relevant toolchains. Section 5 details our experimental results and performance evaluations, highlights the ESP32-S3 deployment, analyzes relevant deployment metrics, and provides a comprehensive comparison with contemporary state-of-the-art methods. Next, Section 6 introduces our end-to-end prototype system that integrates the ESP32-S3 with the WitMotion IMU sensor, elaborating on the decision-making process regarding sensor integration, power consumption, and practical implementation challenges. Section 7 then addresses the limitations of our study and offers guidelines for future research efforts. Section 8 contextualizes our work within the broader challenge of Artificial Intelligence (AI) accessibility such as democratization of advanced AI and global resource inequality. Finally, Section 9

summarizes our contributions and outlines potential directions for future HAR research.

2. Literature review and comparative study

Recent advances in human activity recognition have focused on utilizing resource-constrained devices, such as the MCUs for on-device inference. Our review highlights key studies on classifying Activities of Daily Living (ADL) while addressing different model architectures and optimization strategies.

Daghero et al. (2022a) proposed efficient 1D CNNs for HAR on MCUs, incorporating hyperparameter optimization and model quantization (floating-point, integer, and mixed precision) to reduce power consumption and memory usage. Their previous work (Daghero et al., 2022b) introduced a two-stage hierarchical model combining a Decision Tree (DT) with a 1D CNN, specifically aimed at improving power efficiency. Using the UCI HAPT dataset with a non-overlapping sliding window of 5 s and subject-independent validation, they demonstrated that edge-based HAR solutions on MCU could significantly reduce the strain on cloud resources. Similarly, Ismail et al. (2023) employed an adaptive CNN architecture design for HAR on MCU, further emphasizing the versatility of CNNs in low-power environments. Despite their different approaches, both studies confirm that CNNs are an effective tool for HAR on resource-constrained devices. Although these studies effectively leveraged resource-constrained MCUs for on-device inference using traditional sensors (i.e., accelerometer, gyroscope, and magnetometer), they did not explore the integration of PP sensors. Investigating the incorporation of PP sensor data within HAR frameworks could provide complementary insights, potentially enhancing feature extraction and recognition accuracy. This gap highlights a promising direction for future research aimed at developing more robust and efficient on-device activity recognition systems which our study addresses.

Several studies have explored the use of both overlapping and non-overlapping sliding windows for segmentation in HAR (Dehghani et al., 2019). Non-overlapping windows, as used by Daghero et al., lower computational complexity but may miss finer-grained activity transitions, while overlapping windows capture smoother transitions at the cost of higher resource consumption. In validation, subject-independent strategies, while more challenging, offer better generalization across different users, whereas subject-dependent validation typically results in higher accuracy (Labrador & Yejas, 2013). Moreover, when detecting composite human activities, many studies recommend employing longer window sizes (Chen & Nugent, 2019; Luo et al., 2024). Since composite activities are characterized by a sequence of simpler, individual actions with varying temporal dynamics, a longer window can effectively integrate and contextualize these sub-activities. This extended temporal coverage aids in reducing ambiguity and misclassification that shorter windows might introduce, thereby enhancing the predictive models' ability to discern complex activity patterns. Investigating these longer windows not only supports improved feature extraction but also provides richer contextual information, ultimately contributing to more robust and accurate activity recognition systems.

Besides that, Contoli and Lattanzi (2023) investigated the application of TensorFlow compression techniques to HAR using DL models onto an ESP32. Here the authors demonstrate that it is possible and reasonable to deploy complex DL networks onto MCU with a proper optimization strategy to reduce latency and memory footprint. However, the optimization comes at a cost of predictive accuracy degradation.

Hinton et al. (2015) demonstrated that KD serves as an effective technique for enhancing the performance of lightweight models. By leveraging a highly accurate teacher model, the logits produced by this teacher can function as soft labels for the student model, thereby improving both the accuracy and generalizability of the resulting lightweight model. Previously, Deng et al. (2024) employed KD and data augmentation to train a lightweight HAR model. However, additional research is necessary to understand the limitations and requirements of deploying such a model on resource-constrained devices.

More recent HAR studies, despite focusing on edge deployment and addressing resource constraints, have predominantly covered theoretical analyses, often omitting practical evaluations on resource-constrained devices. For instance, Gonçalves et al. (2024) employed a knowledge distillation approach combined with grid search to develop a lightweight student CNN model for edge deployment, resulting in a model 18 to 42 times smaller than the original teacher model. Although this methodology demonstrates significant reductions in model complexity, the authors did not deploy the compressed model on any edge hardware nor evaluate it within a complete end-to-end system. Similarly, AlMuhaideb et al. (2024) proposed a custom ResLSTM architecture to alleviate the heavy computational load associated with BiLSTM models. Despite achieving higher accuracy and a reduction in the number of parameters, their work also overlooks practical deployment aspects, as it does not extend to full system integration or testing on actual edge devices. While advancing model efficiency is a valuable step forward, practical implementation and deployability remain crucial to ensure that these models meet end-to-end system requirements in real-world scenarios.

Multi-sensor fusion via knowledge distillation has garnered significant interest in the research community, with recent studies successfully integrating data from multiple sensors into a cohesive framework (Mardanpour et al., 2023). However, the use of PP sensors in the context of HAR remains underexplored; to date, no study has systematically examined their effect on activity recognition performance. Moreover, several investigations have largely focused on theoretical performance improvements, neglecting the practical aspects of deployment and on-device performance metrics (Kagiyama & Okita, 2025). Although theoretical advancements are important, it is equally essential to complement these findings with empirical validations, as practical deployment challenges often result in substantial deviations from theoretical expectations. Additionally, state-of-the-art sensor fusion approaches that incorporate attention mechanisms (Muniasamy & Asiri, 2025) demonstrate the potential to adaptively refine feature maps, further underscoring the need for comprehensive evaluations that bridge theory and practice.

3. CAPPIMU dataset and preparation steps

This study employs the CAPPIMU dataset (Luo et al., 2024), a multi-sensor dataset uniquely suited for investigating the interplay between IMU and PP sensor data. To the best of our knowledge, it is the only publicly available dataset that combines synchronized readings from accelerometers, gyroscopes, magnetometers, and PP sensors, addressing a critical gap in wearable sensor research. This dual-modality design aligns directly with our objective of evaluating the role of PP data in KD (see Section 4), making it the natural choice of our study.

The dataset was collected from 30 male subjects aged 20-25 and comprises approximately 30 h of total recording time (60 min per subject), resulting in over 11.9 million data samples after cleaning. This extensive data volume ensures the representativeness of the findings. The 21 captured ADLs are intentionally diverse and are categorized by complexity into simple and composite activities to thoroughly evaluate model performance.

Simple activities, such as *walking*, *jogging*, or *watching TV*, are characterized by relatively uniform and repetitive motion patterns. In contrast, composite activities consist of a sequence of varied sub-actions. For instance, an activity like *washing dishes* involves multiple steps such as scrubbing, rinsing, and placing items on a rack; making it more complex than a simple motion. Other examples of composite activities include *brushing teeth*, *folding clothes*, and *ironing*. This distinction is critical for assessing a model's ability to capture both basic movements and intricate, multi-stage tasks.

Since one of the objectives of this study is to investigate the effect of PP sensor in KD based sensor fusion, we focused on the *insole_wrist_r*

Algorithm 1 Brief CAPPIMU dataset preprocessing pipeline.

Require: Base directory D_{base}, Window size W, Step size S, Train ratio ρ.
Ensure: Training set (X_{train}, y_{train}), Testing set (X_{test}, y_{test}).
  $\mathbf{D} \leftarrow$ LoadSubjectData(D_{base}, "joint/insole_wrist_r.csv") ▷ Load & combine subject CSVs
  $\mathcal{F} \leftarrow$ GetFeatureColumns("insole_l/r(p1-8)", "wrist_r(ax/y/z, wx/y/z, ex/y/z)")
  $\mathbf{D}[\mathcal{F}] \leftarrow$ ZScoreNormalization($\mathbf{D}[\mathcal{F}]$) ▷ Normalize features $\mathcal{F}$
  $\mathbf{D}$['label'] $\leftarrow$ EncodeLabels($\mathbf{D}$['label']) ▷ Map activity strings to integers
  Initialize $\mathbf{X}_{tr}, \mathbf{y}_{tr}, \mathbf{X}_{te}, \mathbf{y}_{te} \leftarrow$ empty lists
  for all subject $s \in$ UniqueSubjects($\mathbf{D}$) **do**
    $\mathbf{D}_s \leftarrow \mathbf{D}$[subject = s].SortByTime()
    Initialize $\mathbf{X}_{tr_s}, \mathbf{y}_{tr_s}, \mathbf{X}_{te_s}, \mathbf{y}_{te_s} \leftarrow$ empty lists
    for all activity $a \in$ UniqueLabels($\mathbf{D}_s$) **do**
      $\mathbf{D}_{s,a} \leftarrow \mathbf{D}_s$[label = a]
      $(\mathbf{X}_{s,a}, \mathbf{y}_{s,a}) \leftarrow$ ApplySlidingWindow($\mathbf{D}_{s,a}, \mathcal{F}, W, S$) ▷ Window data, flatten features, use mode label (highest)
      $(X_{train}, y_{train}), (X_{test}, y_{test}) \leftarrow$ SplitData($\mathbf{X}_{s,a}, \mathbf{y}_{s,a}, \rho$) ▷ Split activity data
      Append (X_{train}, y_{train}) to $(\mathbf{X}_{tr_s}, \mathbf{y}_{tr_s})$
      Append (X_{test}, y_{test}) to $(\mathbf{X}_{te_s}, \mathbf{y}_{te_s})$
    end for
    Append Combine($\mathbf{X}_{tr_s}, \mathbf{y}_{tr_s}$) to $(\mathbf{X}_{tr}, \mathbf{y}_{tr})$ ▷ Combine activities for subject s
    Append Combine($\mathbf{X}_{te_s}, \mathbf{y}_{te_s}$) to $(\mathbf{X}_{te}, \mathbf{y}_{te})$
  end for
  $(X_{train}, y_{train}) \leftarrow$ CombineAll($\mathbf{X}_{tr}, \mathbf{y}_{tr}$) ▷ Combine all subjects
  $(X_{test}, y_{test}) \leftarrow$ CombineAll($\mathbf{X}_{te}, \mathbf{y}_{te}$)
  SaveData("output/path", $X_{train}, y_{train}, X_{test}, y_{test}$) ▷ Save datasets to text files
  return $(X_{train}, y_{train}), (X_{test}, y_{test})$

files, which include IMU data from the right wrist position shown to yield higher accuracy (Luo et al., 2024) paired with synchronized PP sensor data. Data preprocessing involved standard scaling (Pedregosa et al., 2024) to normalize IMU and PP features followed by numerical encoding of activity labels necessary for model training.

To capture temporal dynamics, a sliding window approach was employed using a window size of 300 samples (a six second window) with a 50 % (three second) overlap. This window size configuration was not determined through an empirical search of various window sizes but was a deliberate methodological choice based on two primary factors. Firstly, a longer window is essential for recognizing composite human activities, as it allows the model to capture the complete contextual information inherent in multi-stage tasks (Chen & Nugent, 2019; Luo et al., 2024). Secondly, this specific window size and overlap align with the benchmark parameters recommended by the dataset's authors. Adopting this standard facilitates a fair and direct comparison with existing and future studies that use the CAPPIMU dataset. In addition to that, the 50 % overlap was implemented to maintain continuity between consecutive windows and mitigate potential information loss at the boundaries (Dehghani et al., 2019; Mekruksavanich & Jitpattanakul, 2022). When determining the label for each window, the label with the highest frequency within that window was selected. This technique effectively segments the time-series data into manageable portions for training and testing (see Algorithm 1 for ease of understanding and implementation). To ensure generalizability across subjects while retaining intra-subject variability, we employed a subject-dependent validation strategy, where data from all subjects contributed to both the training and testing phases. The dataset was partitioned into an 80 %–20 % train-test split, with 15 % of the training set allocated for validation (see Fig. 1).

4. Proposed methods

This study employs Knowledge Distillation (Chitty-Venkata et al., 2023) to transfer dark knowledge from a complex teacher model, DualCrossTAP (we named the model DualCrossTAP due to its two-branch architecture, cross-sensor attention, and temporal attention pooling), to lightweight CNN student models, enabling efficient deployment on highly resource-constrained MCUs while prioritizing accuracy and execution speed. The teacher model integrates dual-branch architecture with cross-sensor attention and residual connections to enhance feature representation, while the student models adopt a minimalist architecture optimized for MCU compatibility. The high-level model architectures and overall methods are outlined in Fig. 2. Below, we detail the design principles and rationale of the teacher and student models.

4.1. Teacher model design

The teacher model, DualCrossTAP, is engineered as a two-branch (dual) architecture to process multimodal data from IMU and PP sensors. The main motivation for such design is the complementary nature of IMU and PP sensors. Extending the analysis conducted by Luo et al. (2024), we observed that IMU was particularly good at detecting hand-related movements such as *brushing_teeth*, *use_computer*, etc. while PP was good at detecting leg-centric activities such as *walk*, *run*, *cycle*, etc. Hence, a two-branch architecture (Moreau et al., 2024) is ideal to independently extract features from IMU and PP sensors.

For feature extraction within each branch, a series of enhanced *Conv* blocks (see Fig. 2) are utilized. Each block consists of a 1D convolutional layer (Guo et al., 2024a; He & Guo, 2024), followed by batch normalization and a ReLU activation function. A residual connection is incorporated by adding the input of the block to its output; a 1x1 convolution is applied to the shortcut if the input and output channel dimensions do not match. This convolutional structure is designed to learn hierarchical spatial features from the sensor data. Max pooling layers are interspersed between the convolutional blocks to reduce the dimensionality of the feature maps and provide a degree of translational invariance.

Following the convolutional feature extraction, each branch employs a transformer encoder block (Vaswani et al., 2023). This block consists of a multi-head self-attention mechanism followed by a feed-forward network. Residual connections and layer normalization are applied after both the attention and feed-forward sub-layers. While Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) units, and Gated Recurrent Units (GRUs) are commonly used for sequence modeling in HAR due to their proficiency with temporal dependencies and imbalanced datasets (Leite et al., 2024; Sonata & Heryadi, 2024), we opted

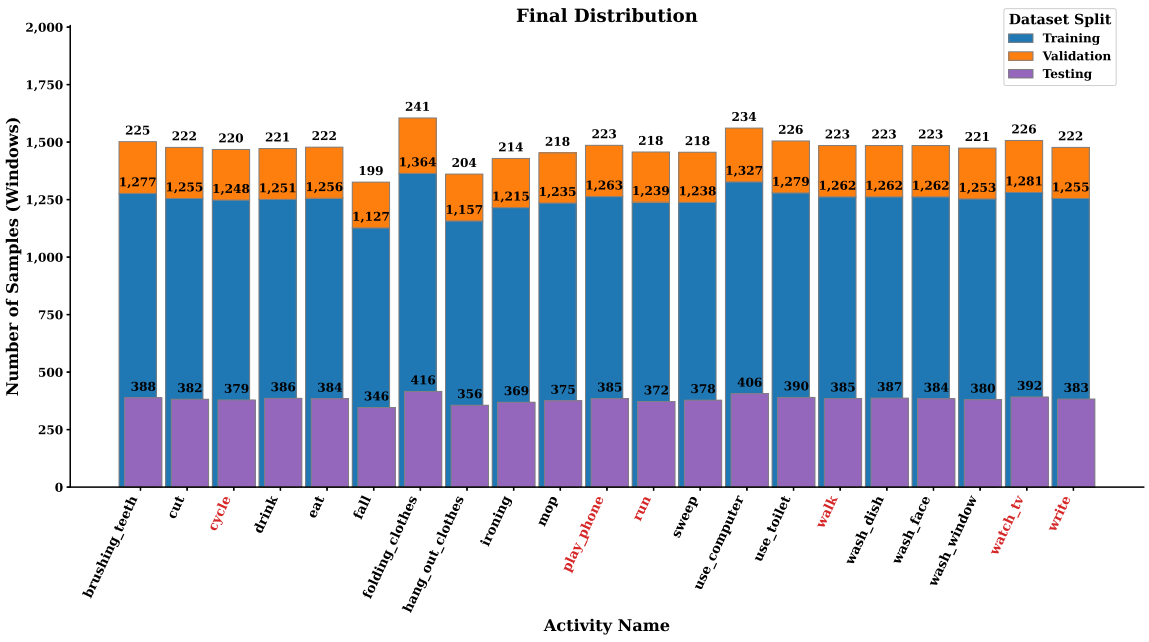


Fig. 1. Shows the final train, test, and validation distribution of CAPPIMU dataset after preprocessing. Activity names colored in red represent simple type activities, while black-colored names represent composite type activities.

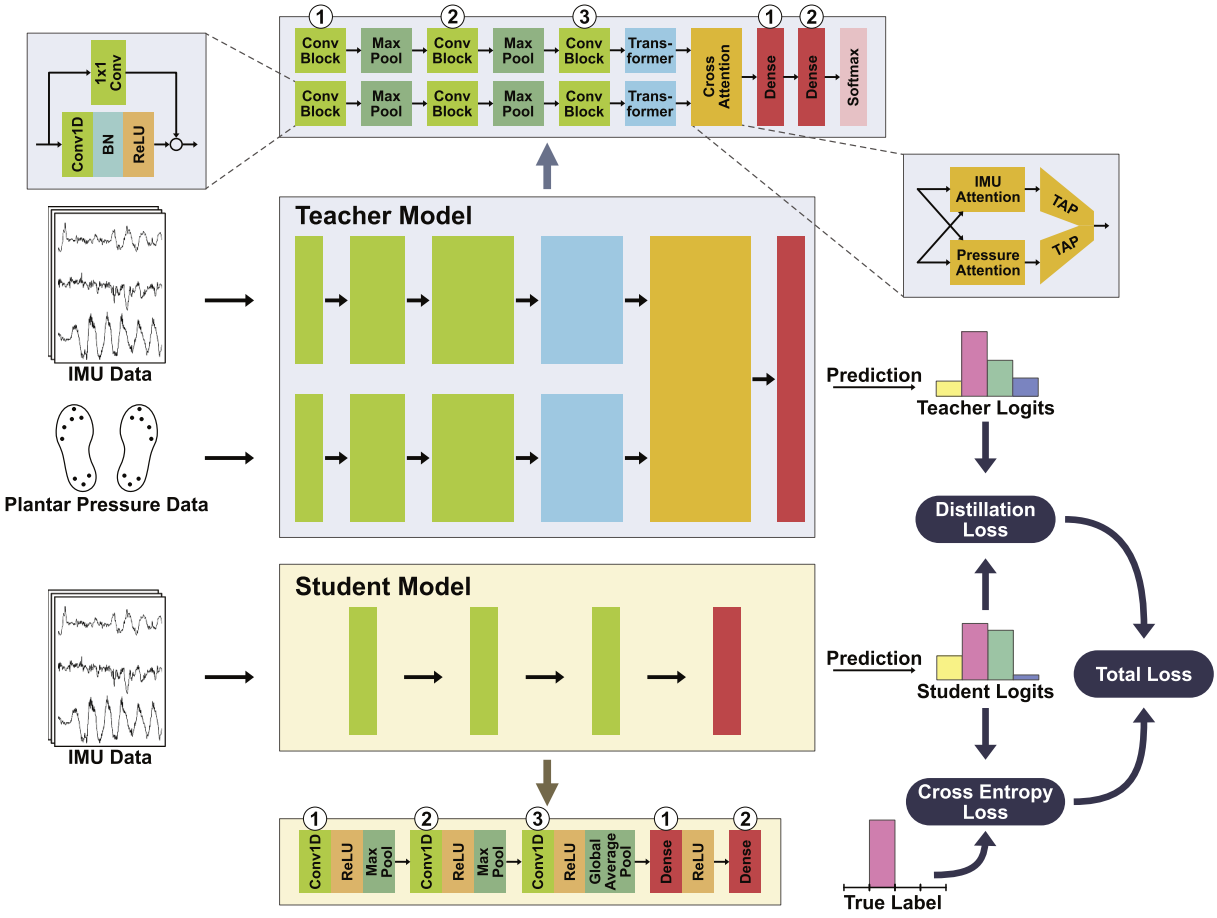


Fig. 2. Illustrates the architecture of our DualCrossTAP teacher and optimized CNN student models, alongside their knowledge distillation workflow.

for transformer layers. This decision was based on two primary factors: firstly, the CAPPIMU dataset exhibits a relatively balanced distribution of activity labels (see Fig. 1), mitigating a key advantage of recurrent architectures in handling class imbalance. Secondly, transformers can leverage parallel processing capabilities of modern accelerators like graphics processing units more effectively than sequential recurrent models (Zeyer et al., 2019), leading to significantly faster training times. Furthermore, in scenarios with balanced datasets, transformer architectures often demonstrate superior performance compared to their recurrent counterparts (i.e., RNN, LSTM, GRU) (Leite et al., 2024). It is acknowledged that transformer layers are typically more computationally expensive than convolution layer, which can be a significant consideration for resource-constrained MCUs. However, in the context of this study, this computational overhead is permissible for the teacher model, as its primary role is to serve as a source of comprehensive knowledge.

After the individual branch processing, cross-modal attention is applied to allow interaction between the IMU and PP features. Specifically, a multi-head attention mechanism is used where IMU features attend to PP features and vice versa, with the original features being added to their attended counterparts to enrich the representations. Finally, to aggregate the temporal information from the processed sequences of each modality, a temporal attention pooling layer is utilized (Gan & Zhang, 2023). This layer learns to assign different weights to different time steps in the sequence, allowing the model to focus on the most informative segments for the activity recognition task. The attention weights, α_t, for each time step t of an input sequence $X = (x_1, x_2, \ldots, x_T)$ are calculated, using a dense layer with a softmax activation, and then used to compute a weighted sum of the input features. The output, c, can be represented as:

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^{T} \exp(e_k)} \tag{1}$$

where $e_t = \text{score}(x_t)$ is a learnable score corresponding to the input feature x_t at time step t, e_k represents the same learnable score for the input feature at time step k within the sum, both computed by a small neural network (i.e., a dense layer with a tanh activation), and T is the total number of time steps in the input sequence $X = (x_1, x_2, \ldots, x_T)$.

The final context vector c is then:

$$c = \sum_{t=1}^{T} \alpha_t x_t \tag{2}$$

This mechanism allows the model to dynamically emphasize salient temporal features from both the IMU and PP sensor streams before they are concatenated and passed to subsequent dense layers for final classification. The detailed architecture and parameters of the teacher DualCrossTAP model are given in Appendix B. This design yields superior accuracy over existing state-of-the-art models, as validated in Section 5.

4.2. MCU and toolchain selection

The ESP32-S3 DevKit C1 (Systems, 2023) was selected as the target MCU due to its popularity, balance of computational resources, cost, and compatibility with TinyML workflows. This board features a dual-core Xtensa LX7 processor (240 MHz), 512 KiB Static Random Access Memory (SRAM), and 8 MiB Flash, enabling efficient deployment of lightweight models while accommodating memory-intensive operations. To contextualize this choice, Table 1 compares alternative MCUs, emphasizing hardware constraints critical for TinyML deployment. Crucially, the selected MCU directly informs the design constraints for the student models; their architectures must be meticulously optimized to ensure that the final compiled binary and runtime memory footprint remains within the available SRAM and Flash memory. This resource-aware design is paramount for successful on-device deployment.

Similarly, PlatformIO (PlatformIO, 2024) was chosen for its cross-platform board management, enabling seamless transitions between MCUs (e.g., ESP32 to STM32) via unified dependency resolution. The Arduino framework (PlatformIO, 2025) further ensures portability as its hardware-agnostic Application Programming Interfaces (APIs) allow researchers to adapt this work to other MCUs with minimal code changes, leveraging widespread vendor support. This toolchain combination prioritizes reproducibility while addressing the computational limits of edge devices.

4.3. Student model designs

The foremost consideration in designing student models for deployment on resource-constrained MCUs such as the ESP32-S3 is adherence to strict hardware limitations. This necessitates a meticulous selection of network layers and operations, with particular attention given to the resultant total Flash and SRAM consumption. While architectures like RNNs and LSTMs networks are proficient in modeling temporal dependencies (Guo et al., 2025a) inherent in sequential data like IMU and PP signals, their typically substantial computational and memory requirements often render them unsuitable for direct deployment on such severely constrained hardware. The overhead associated with their recurrent connections and gating mechanisms can easily exceed the available resources on MCUs. Consequently, for this study, CNN-based architectures were prioritized, as they can offer a more favorable trade-off between feature extraction capabilities for time-series data and resource efficiency. Besides that, TensorFlow Lite Micro (TFLM) provides optimized kernels for convolution-related operations.

In this context, two distinct CNN-based student models were developed. The first model was engineered with the primary goal of maintaining high classification accuracy, while the second was optimized for superior performance metrics, including inference speed and power efficiency, on the target MCU. The first student model, designated CNN-S1, is conceived as a compact and efficient CNN, specifically tailored for deployment on the ESP32-S3 and similar MCUs (see Section 4.2), whose resource constraints were detailed previously. The principal objective underpinning CNN-S1 is the creation of a lightweight model capable of performing HAR effectively on-device. A secondary, yet crucial, objective is its amenability to knowledge distillation from a more complex teacher model, DualCrossTAP (see Section 4.1), while striving to retain comparable accuracy and precision. To meet these objectives, the architecture of CNN-S1 is intentionally streamlined to minimize its computational and memory footprint. It primarily comprises a sequence of 1D convolutional layers (as illustrated in Fig. 2), which are particularly adept at extracting salient temporal features from time-series IMU sensor data. These are complemented by max pooling layers to systematically reduce dimensionality and computational load, which is an essential characteristic for low-compute devices. The selection of activation functions, alongside the network's depth and breadth, was carefully calibrated to strike an optimal balance between predictive performance and resource

Table 1
Comparison of MCUs that are similar to ESP32-S3 and if desired, could also be used instead of ESP32-S3.

Board	Processor	Dimension (mm)	Price ($)	SRAM/Flash
esp (2023)	Xtensa LX7 (240 MHz)	25.5 × 18.0	5–8	512 KiB / 8 MiB
Nano 33 BLE Sense Rev2 (2025)	Cortex-M4 (64 MHz)	18 × 45	25	256 KiB / 1 MiB
STS (2025)	Cortex-M7/M4 (480 MHz)	25.5 × 25.5	15–20	1 MiB / 2 MiB
Ras (2025)	Cortex-M0 + (133 MHz)	21 × 51	6	264 KiB / 2 MiB
Nordic nRF5340 (2025)	Cortex-M33 (128 MHz)	7 × 7	10–12	512 KiB / 1 MiB

utilization. Subsequently, global average pooling is employed to further curtail computational demands. The terminal stage of CNN-S1 consists of dense layers, which are tasked with classifying the extracted features into distinct 21 activity categories. The overall structure is meticulously designed to ensure that the underlying NN Operations (OPS) (see TensorFlow, 2024), especially when converted for TFLM, are optimized for minimal memory and computational overhead. This design facilitated activity classification with high accuracy while maintaining moderate on-device performance (see Section 5).

The second student model, named CNN-S2, shares the foundational architecture (see Appendix C) of CNN-S1 but incorporates a strategic modification aimed at significantly enhancing inference time and reducing its flash memory usage. Specifically, all standard 1D convolutional blocks within CNN-S1 were replaced with 1D depthwise separable convolutional blocks (Howard et al., 2017). This architectural adjustment yielded a substantial reduction in the total parameter count by over 42 %, leading to markedly lower Flash memory consumption. However, this optimization introduced a trade-off, resulting in increased peak SRAM usage during inference and a discernible reduction in classification accuracy when compared to the CNN-S1 model as detailed in Section 5.

4.4. Knowledge distillation method

This study employs a standard response-based knowledge distillation technique (Chen et al., 2023; Passalis et al., 2022; Yang et al., 2023). The primary objective is not to introduce a novel KD algorithm but rather to adapt and apply established KD principles to a custom requirement: exploring the efficacy of KD-based sensor fusion for HAR using IMU and PP data, specifically targeting resource-constrained MCUs. This investigation addresses a notable gap in the existing HAR literature, as the fusion of IMU and PP sensor modalities through KD for deployment on MCUs has not been explored. The focus on MCUs is particularly pertinent, as these are environments where efficient sensor data utilization and model compression are paramount.

The core of our KD-based sensor fusion strategy is as follows: The teacher model, DualCrossTAP (detailed in Section 4.1), is initially trained using data from both IMU and PP sensors. Subsequently, during the knowledge distillation process, the student model is trained using input from only a single sensor modality. This approach allows the student model to implicitly learn and benefit from the knowledge of the excluded sensor modality, which is distilled from the more comprehensive understanding of the teacher model. Given the two sensor types, two distinct fusion pathways were investigated:

I. **PP to IMU Fusion**: The student model is trained primarily with IMU sensor data, with knowledge from the PP sensor being distilled from the teacher.
II. **IMU to PP Fusion**: The student model is trained primarily with PP sensor data, with knowledge from the IMU sensor being distilled from the teacher.

Despite exploring both pathways, the strategy prioritizing active IMU data for the student model emerges as the more pragmatic choice for highly resource-constrained devices. This preference is substantiated by several factors. Firstly, the input data dimensionality associated with the IMU sensor is considerably smaller than that of the PP sensor. This directly translates to a reduced input buffer size and a lower number of MFLOPs for the student model. For instance, considering a six-second window and a 50 Hz sampling rate, an IMU with 9 axes (3-axis accelerometer, 3-axis gyroscope, 3-axis magnetometer) requires an input buffer for $9 \times 50 \times 6 = 2700$ float (32-bit) values, equating to 10.8 KB. In contrast, a PP sensor system with 16 pressure points (eight pressure points per leg) necessitates an input buffer for $16 \times 50 \times 6 = 4800$ values, or 19.2 KB. Secondly, empirical evidence from existing state-of-the-art models suggests that IMU sensors are inherently capable of distinguishing a broader range of activities more effectively than PP sensors, exhibiting an accuracy advantage of over 12 % in some cases (Luo et al., 2024). Thirdly, the ergonomic aspects and wearability of the sensors are critical for ensuring user compliance and continuous data acquisition. The IMU sensor, positioned on the right wrist, offers superior comfort and practicality. Its unobtrusive nature allows for consistent wear across a wide range of daily activities, including extended periods of rest and sleep, which is vital for comprehensive and uninterrupted monitoring. On the other hand, the PP sensor, situated on the leg, presents practical limitations for continuous, long-term use. Its placement can be cumbersome and may cause discomfort, particularly during sleep or sedentary states (Qiu et al., 2019), potentially leading to inconsistent data collection and reduced user adherence. This significantly curtails its suitability for applications requiring persistent, round-the-clock monitoring. Consequently, for deployment on resource-constrained MCUs, selecting the IMU as the primary active sensor for the student model, augmented by distilled knowledge from the PP sensor (PP to IMU Fusion), presents an optimal approach. Nevertheless, for completeness and to provide valuable insights for future research, experiments involving the distillation from IMU to a PP-reliant (IMU to PP Fusion) student model were also conducted. The results, presented in Section 5, further corroborate the relative inferiority of the PP sensor as the sole active input in this distilled sensor fusion context.

The employed KD methodology is visually outlined in Fig. 2. In our primary configuration, the DualCrossTAP teacher model processes inputs from both IMU and PP sensors. Conversely, the student CNN model receives only IMU sensor data as direct input, ensuring a lightweight footprint (see Algorithm 2). This setup facilitates the implicit transfer of knowledge extracted from plantar pressure data by DualCrossTAP to the student CNN, even though the student does not directly process PP data. The knowledge transfer is mathematically formalized using a temperature-scaled Kullback-Leibler (KL) divergence (Hinton et al., 2015) between the logits of the teacher and student models. For a given input $\mathbf{x}$, the total loss function $\mathcal{L}_{\text{total}}$ is a weighted sum of the standard cross-entropy loss and the distillation loss:

$$\mathcal{L}_{\text{total}} = \alpha \underbrace{\mathcal{L}_{\text{CE}}(y, \mathbf{z}_s)}_{\text{Cross-Entropy}} + (1-\alpha) \underbrace{T^2 \cdot D_{\text{KL}}(\sigma(\mathbf{z}_t/T) \parallel \sigma(\mathbf{z}_s/T))}_{\text{Distillation}} \tag{3}$$

In this equation, $\mathbf{z}_t$ and $\mathbf{z}_s$ represent the logits of the teacher and student models, respectively. The temperature parameter T is used to soften the probability distributions derived from the logits, thereby controlling the granularity of the knowledge transferred. The hyperparameter α balances the contribution of the hard target labels (via cross-entropy) and the soft targets from the teacher (via distillation loss). The KL divergence term, D_{KL}, is defined as:

$$D_{\text{KL}} = \mathbb{E}\left[\sum_{i=1}^{C=21} \sigma\left(\frac{\mathbf{z}_t}{T}\right)_i \log\left(\frac{\sigma(\mathbf{z}_t/T)_i}{\sigma(\mathbf{z}_s/T)_i}\right)\right] \tag{4}$$

where C is the number of classes (21 in this study), and $\sigma(\cdot)$ denotes the softmax function applied to the temperature-scaled logits. This formulation encourages the student model's output distribution to mimic that of the teacher model.

5. Experimental results and analysis

This section details the experimental methodology used to evaluate the performance of our composite HAR system on the resource-constrained device (ESP32-S3). The analysis covers model training, optimization techniques, and deployment on the ESP32-S3 board. The experiments were conducted to assess not only classification accuracy but also computational efficiency, including memory (SRAM) and Flash usage, inference latency, and power consumption. We focused on balancing model performance with the inherent hardware limitations of the ESP32-S3, aiming for an optimized NN solution suitable for real-world, low-power applications utilizing our developed end-to-end prototype.

Algorithm 2 High-level PP to IMU knowledge distillation training process.

Require: Pre-trained teacher model $\mathbf{T}$ (accepts IMU and PP data); Student model $\mathbf{S}$ (accepts IMU data only, parameters can be uninitialized or pre-trained); Training dataset $D_{train} = \{((X_i^{\text{IMU}}, X_i^{\text{PP}}), Y_i)\}_{i=1}^{N}$ where X_i^{IMU} is IMU data, X_i^{PP} is Plantar Pressure data, and Y_i are true labels; Temperature parameter $\tau > 1$; Loss weighting factor $\alpha \in [0, 1]$.

Ensure: Trained student model $\mathbf{S}_{trained}$.

```
Initialize parameters of student model S (if not already pre-trained).
for all training epoch e do
    for all batch ((X_b^IMU, X_b^PP), Y_b) in D_train do
        Z_T ← T(X_b^IMU, X_b^PP)          ▷ Compute teacher's logits using both IMU and PP data
        Z_S ← S(X_b^IMU)                  ▷ Compute student's logits using only IMU data
        P_T^soft ← Softmax(Z_T/τ)         ▷ Calculate teacher's soft targets using temperature τ
        P_S^soft ← Softmax(Z_S/τ)         ▷ Calculate student's soft predictions using temperature τ
        L_hard ← CrossEntropyLoss(Softmax(Z_S), Y_b)   ▷ Student's loss against true labels
        L_soft ← τ^2 · CrossEntropyLoss(P_S^soft, P_T^soft)   ▷ Student's loss against teacher's soft targets (scaled)
        L_total ← α · L_hard + (1 − α) · L_soft   ▷ Combine hard and soft losses
        Update parameters of S using gradient of L_total
    end for
end for
S_trained ← S
return S_trained
```

5.1. Experimental setup

Our experiments were conducted using a high-performance personal computer equipped with a Ryzen 9 5900X CPU, RTX 3090 (24 GB) Graphics Processing Unit (GPU), and 128 GB of memory, running Pop!_OS 22.04 LTS with the Linux kernel 6.9.3. This setup provided the necessary computational infrastructure for training and deploying ML models. We used Python 3.10 due to its stability, alongside the Scikit-Learn library 1.6 (Pedregosa et al., 2011) and the TensorFlow 2.17 framework (Abadi et al., 2015) for dataset preprocessing, model development, training-testing, and optimization.

Furthermore, we used PlatformIO core version 6.1.16 (PlatformIO, 2024) with CLion (JetBrains, 2024) for firmware development as PlatformIO simplifies the process of managing vendor-specific toolchains, libraries, and dependencies. Consequently, the ESP32-S3 Dev Kit C1 (Systems, 2023) is a powerful MCU, with a 32-bit, dual-core Tensilica Xtensa LX7 processor running at 240 MHz, and 320 KiB of SRAM with 8 MiB of Flash memory. Primarily, this MCU was selected due to its popularity, low cost, and power efficiency (see Section 1). We tested and collected the inference performance of ESP32-S3 using the testing split transmitted via Universal Asynchronous Receiver/Transmitter (UART) (Arduino, 2024), simulating real-time data handling. Next, the power consumption was measured utilizing a KWS-MX18L Universal Serial Bus (USB) power monitor (Keweisi, 2025). This specific device was selected due to its suitability for long-term power consumption logging and its pass-through USB design. The latter feature significantly streamlines the development workflow by allowing simultaneous serial communication (for data transmission and debugging) and code deployment to the MCU without requiring disconnection or alternative measurement setups, such as traditional USB multimeters, which often preclude concurrent data operations.

5.2. Model training and CNN-S1 ESP32-S3 deployment

The DualCrossTAP model was trained for 50 epochs utilizing the training and validation splits detailed in Section 3. A batch size of 32 was employed, and the AdamW optimizer (TensorFlow, 2025) was selected with a learning rate of 0.0001 and a weight decay of 0.0001. To mitigate potential overfitting and enhance generalization, ModelCheckpoint callbacks and ReduceLROnPlateau learning rate scheduling strategies were implemented (Team, 2025).

As anticipated, the DualCrossTAP model demonstrated exemplary performance, achieving an accuracy of 99.32 % and an F1 Score of 99.33 %, thereby outperforming existing state-of-the-art models in this dataset. A more granular analysis of DualCrossTAP's performance across each of the 21 activities revealed consistently high F1 scores. A minor exception was observed for the *play_phone* activity, which registered a slightly lower F1 score of 97 %, while the majority of other activities achieved scores of 98 % or higher. To rigorously validate the model's robustness and ensure that the observed performance was not a result of overfitting to the specific train-test split, comprehensive 5-Fold and 10-Fold cross-validation procedures were conducted. During these cross-validation experiments, careful attention was paid to preserving temporal dependencies within the data and preventing data leakage between folds. The results of these cross-validation analyses are presented in Table 2 and compared with the initial train-test split results. The cross-validation results consistently affirm the effectiveness of the DualCrossTAP model and its dual-branch architecture for robust IMU and PP sensor fusion. The model demonstrates superior performance on both hand-centric and leg-centric activities, underscoring the significant contribution of the plantar pressure sensor data, a modality often overlooked in previous HAR studies. Furthermore, these findings validate the efficacy of the transformer layer within our architecture, particularly when applied to a dataset with balanced class distributions, as evidenced by the model's capacity for effective learning and generalization. Quantitatively, the DualCrossTAP model comprises a total of 690,711 parameters (2.63 MiB in size) and requires 67.90 MFLOPs for inference. This analysis provides insight into the computational requirements of the teacher model, which serves as the basis for knowledge distillation to the more resource-constrained student models.

Table 2
Comparison of DualCrossTAP models' performance across different validations.

Validation	Accuracy	F1 Score (macro)	Precision (macro)	Recall (macro)
Train Test Split	99.32 % ± 0.35	99.33 % ± 0.35	99.34 % ± 0.27	99.34 % ± 0.33
5-Fold	99.60 % ± 0.27	99.56 % ± 0.16	99.58 % ± 0.14	99.56 % ± 0.17
10-Fold	99.63 % ± 0.25	99.69 % ± 0.16	99.65 % ± 0.14	97.79 % ± 0.16

Next, for the KD-based PP to IMU sensor fusion, the CNN-S1 student model was trained for 1000 epochs utilizing only IMU data (see Algorithm 2). The training maintained the same optimizer parameters as the teacher model, with a temperature $T = 3.0$ and a loss component weight $\alpha = 0.75$, as defined in Eqs. (3) and (4). The resulting distilled model, CNN-S1, achieved a notable performance with 98.35 % accuracy and a 98.36 % F1 score. This demonstrates significant performance retention compared to training the student model without

Table 3
Comparison of our optimized CNN-S1 model against state-of-the-art approaches, considering only the PP sensor. To compute *Total Parameters*, *Trainable Parameters*, and *MFLOPs* from the study (Luo et al., 2024), we referred to their cited GitHub repository. As their models are implemented in PyTorch, we employed *ptflops* (Contributors, 2024a) and *summary* (Contributors, 2024b) for MFLOPs and model summary calculations.

Study	Model	Total Parameters	Trainable Parameters	MFLOPs	Compute	F1 Score
Luo et al. (2024)	DeepConvLSTM	787,669	787,669	988.48	N/A	63.6 %
	FCN	194,005	194,005	18.24		85.8 %
	DCNN	23,094,975	23,094,975	901.59		76.0 %
	TPN	96,437	96,437	49.45		64.2 %
	Shallow-LSTMs	655,573	655,573	914.26		56.9 %
	CNN-TA	1,107,786	1,107,786	118.80		75.4 %
This Study	**CNN-S1**	**51,701**	**51,701**	**6.55**	**ESP32-S3**	**85.90 %**

Table 4
Comparison between DualCrossTAP and student CNN-S1 model. The superscript 1 represents KD-based PP to IMU sensor fusion, and superscript 2 represents KD-based IMU to PP sensor fusion in the *Model* column.

Model	Sensor	Accuracy	F1 Score (macro)	Total Parameters	Trainable Parameters	MFLOPs
DualCrossTAP	IMU & PP	99.32 % ± 0.35	99.33 % ± 0.35	690,711	688,279	67.90
CNN-S1 without KD	IMU	95.63 % ± 0.77	95.56 % ± 0.86	51,029	51,029	6.14
CNN-S1 with KD^1	IMU	98.35 % ± 0.21	98.36 % ± 0.18			
CNN-S1 without KD	PP	82.28 % ± 0.32	82.37 % ± 0.25	51,701	51,701	6.55
CNN-S1 with KD^2	PP	86.12 % ± 0.26	85.90 % ± 0.11			

teacher guidance (see Table 4) and surpasses existing state-of-the-art models. Furthermore, the CNN-S1 model comprises a total of 51,029 parameters (approximately 199.33 KiB). This represents a compression factor exceeding 13× in terms of size compared to the DualCrossTAP teacher model, with only a minimal reduction in accuracy. The computational cost is also significantly reduced, with the CNN-S1 model requiring only 6.14 MFLOPs, making it substantially less computationally intensive than the teacher model.

Similarly, we conducted experiments on KD-based IMU to PP sensor fusion using the aforementioned temperature and loss component settings. As anticipated and discussed in Subsection 4.4, the CNN-S1 model in this configuration exhibited considerably worse performance, achieving 86.12 % accuracy and an 85.90 % F1 Score. With a total parameter count of 51,701 (201.96 KiB) and 6.55 MFLOPs, this strategy is significantly less effective than the PP to IMU fusion approach, primarily due to the substantial accuracy drop of over 12 %, rendering it impractical for deployment in scenarios where high accuracy is critical. Nevertheless, it is worth noting that even with only the PP sensor data, this KD-based sensor fusion approach still outperforms existing state-of-the-art models by a small margin (see Table 3). The comprehensive comparison between the DualCrossTAP teacher model and the CNN-S1 student model, including the performance of both explored sensor fusion strategies, is presented in Table 4.

Before deploying the CNN-S1 model on the ESP32-S3, we evaluated several quantization methods to further optimize its performance while accommodating the device's strict resource constraints. Table 5 provides a detailed comparison of these techniques in terms of accuracy, F1 Score, precision, model size, and deployability. The Float-16 quantization achieves nearly lossless accuracy, precision, and F1 Score while reducing the model size by roughly 50 % compared to the default 32-bit float configuration. However, as the ESP32-S3 does not natively support 16-bit floating point operations and TFLM offers no accommodation for this format; the anticipated reduction cannot be fully exploited. Similarly, mixed precision quantization maintains competitive performance (approximately 97.6 % accuracy) yet still relies on 16-bit floats. Furthermore, the use of heterogeneous numerical representations (16-bit float and 8-bit integer) across different layers incurs additional memory overhead for quantization and dequantization OPS, making this method less practical despite achieving a model size reduction of over 67 %. In contrast, 8-bit integer quantization significantly improves inference latency by leveraging optimized 8-bit integer kernels available in TFLM. However, the associated accuracy drop of roughly 6 % precludes its adoption. Hence, the unquantized CNN-S1 model remains the preferred candidate for deployment on the resource-constrained ESP32-S3.

The finalized CNN-S1 model was converted to TensorFlow Lite format with default optimizations, transpiled to a C header array using xxd tool (Kormanyos, 2024). We then imported the transpiled C header file in the PlatformIO project, resolved the necessary dependencies for TFLM (for ease of use and deployment, we used our developed PlatformIO library (Inmoresentum, 2025)), substituted the appropriate OPS, and deployed via PlatformIO to the ESP32-S3 Dev Kit C1.

The deployed CNN-S1 model attains 98.355 % accuracy and 98.365 % F1 Score, thereby meeting our performance expectations. Furthermore, the model demonstrates an inference latency of 1,836,288 microseconds approximately (see Table 6 for detailed breakdown) and exhibits a peak SRAM usage of 62 KB during operation, with a total model footprint of 209.108 KB. On the ESP32-S3 platform, the overall memory consumption is 91.636 KB of SRAM and 552.649 KB of Flash. Within the SRAM allocation, the CNN-S1 model uses 62 KB while the input buffer requires 10.8 KB; the remaining memory is apportioned to system components such as FreeRTOS and UART etc. Similarly, in Flash memory, the model occupies 209.108 KB, with the rest being utilized by FreeRTOS, UART, string literal, and additional system functionalities. Additionally, the ESP32-S3 operated at a constant 240 MHz without engaging DFS, and an average power consumption of 210 mW was recorded throughout the testing phase. It is important to note that the reported power consumption corresponds to the entire ESP32-S3 Dev Kit C1 (board), rather than solely to the chip itself (Systems, 2025a,c).

In typical use-case scenarios, the battery-powered MCU must balance low power consumption with acceptable performance to extend battery life. To identify the optimal configuration for the ESP32-S3, we evaluated both fixed frequency settings (ranging from 10 to 240 MHz) and DFS mode (see Table 7). In our experiments, we measured CNN-S1 inference latency, power consumption, and CPU temperature using

Table 5
Comparison of various quantization methods applied to the student CNN-S1 model.

Method	Accuracy	F1 Score (macro)	Precision (macro)	CNN-S1 Size	Deployable
None	98.35 %	98.36 %	98.36 %	209.108 KB	✓
Float-16	98.34 %	98.33 %	98.32 %	108.116 KB	✗
Mixed Precision	97.60 %	97.63 %	97.65 %	67.000 KB	✗
Integer-8	92.23 %	91.38 %	93.05 %	67.016 KB	✓

Table 6
Per-layer and per-operation execution time for CNN-S1 on ESP32-S3, running at a constant CPU frequency of 240 MHz. Execution times were measured as the average of 5000 randomly sampled inference runs, with detailed per-operation latency profiles collected during benchmarking. During the conversion from TensorFlow to TFLite format, the converter replaced all 1D convolutions with 2D convolutions, as native support for 1D convolution is currently unavailable in TFLM. Since 1D convolution is a special case of 2D convolution (Contributors, 2021; Developers, 2024), the TFLite converter introduces a *Reshape* layer to transform 1D data into 2D format, ensuring compatibility. Finally, the last row reports peak SRAM utilization of 62 KB and the total execution time (inference latency) of 1,836,271 micro seconds.

Layer	Operation	Operation Execution Time (µs)	Layer Total Execution Time (µs)
Reshape	SHAPE	7	83
	STRIDED_SLICE	32	
	PACK	9	
	RESHAPE	35	
Convolution	CONV_2D	154,200	154,200
Max Pool	MAX_POOL_2D	6035	6035
Convolution	CONV_2D	517,005	517,005
Max Pool	MAX_POOL_2D	6011	6011
Convolution	CONV_2D	1,137,845	1,137,845
Global Average Pooling	MEAN	11,927	11,927
Dense	FULLY_CONNECTED	2672	2672
Output (Dense)	FULLY_CONNECTED	446	493
	SOFTMAX	47	
	62 KB (SRAM)		**1,836,271**

Table 7
Performance evaluation of the ESP32-S3 using the CNN-S1 student model at various CPU frequencies. In *DFS* mode, the CPU dynamically operates between a minimum of 80 MHz and a maximum of 240 MHz. The displayed CPU temperatures were recorded using the ESP32-S3's built-in sensor.

Mode	CPU Frequency (MHz)	CNN-S1 Inference Latency (µs)	Power Consumption (mW)	CPU Temperature (K)
Dynamic Frequency Scaling (DFS)	min: 80 and max: 240	1836941	212	306.70
Fixed/Constant	240	1,836,278	210	306.75
	160	2,705,835	130	305.75
	80	5,354,446	101	305.70
	40	10,959,680	64	304.75
	20	23,005,809	56	303.75
	10	50,918,089	48	304.75

the onboard temperature sensor to assess performance and determine whether additional cooling might be required.

Table 7 shows that as the CPU frequency decreases, the inference latency increases markedly. For example, while the fixed 240 MHz configuration produces an inference latency of approximately 1.84 s with a power draw of about 210 mW, operating at 160 MHz increases the latency to roughly 2.7 s while reducing power consumption to 130 mW. Conversely, frequencies below 80 MHz result in dramatically higher latencies exceeding 10 s at 40 MHz and also disable key functionalities such as Wi-Fi and Bluetooth, making these configurations impractical. Notably, when using DFS, the CPU dynamically ramps up to the maximum frequency (240 MHz) during inference, yielding performance identical to the constant 240 MHz frequency. As a result, considering both power efficiency and inference latency, the fixed 160 MHz configuration provides the most balanced performance.

5.3. CNN-S2 training and ESP32-S3 deployment

Consistent with the training methodology employed for the CNN-S1 student model in the PP to IMU sensor fusion context, the CNN-S2 student model was also trained for 1000 epochs. This training utilized identical optimizer parameters, temperature settings, and loss components as its predecessor. As hypothesized in Subsection 4.3, the CNN-S2 model demonstrates superior parameter efficiency compared to CNN-S1. Specifically, CNN-S2 comprises 30,288 parameters (approximately 118.31 KiB) and requires 2.19 MFLOPs. This configuration represents a

Table 8
Comparisons between CNN-S1 and CNN-S2 student models along with and without KD-based PP to IMU sensor fusion.

Model	Accuracy	F1 Score (macro)	Total Parameters	Trainable Parameters	MFLOPs
CNN-S1 without KD	95.63 % ± 0.77	95.56 % ± 0.86	51,029	51,029	6.14
CNN-S1 with KD	98.35 % ± 0.21	98.36 % ± 0.18			
CNN-S2 without KD	94.22 % ± 0.55	94.21 % ± 0.49	30,288	30,288	2.19
CNN-S2 with KD	97.68 % ± 0.39	97.68 % ± 0.35			

Table 9
Comparison of various quantization methods applied to the student CNN-S2 model.

Method	Accuracy	F1 Score (macro)	Precision (macro)	CNN-S2 Size	Deployable
None	97.68 %	97.68 %	97.92 %	127.984 KB	✓
Float-16	97.66 %	97.67 %	97.92 %	68.952 KB	✗
Mixed Precision	96.49 %	96.43 %	96.61 %	50.840 KB	✗
Integer-8	89.92 %	89.53 %	91.18 %	50.856 KB	✓

significant reduction of 20,741 parameters relative to CNN-S1. More notably, it achieves substantial compression of 657,991 parameters (an approximate 23× reduction) when compared to the teacher DualCrossTAP model. Furthermore, the reduction in the computational load, evidenced by a decrease of nearly 4 MFLOPs compared to CNN-S1, offers a considerable improvement in terms of inference latency. However, these enhancements in efficiency are accompanied by a marginal trade-off in performance. The CNN-S2 model achieves accuracy and F1 Scores of 97.681 % and 97.685 % (macro), which, on average, are approximately 0.7 % lower than those recorded for CNN-S1. Despite this noticeable decrease, CNN-S2 successfully learned from the teacher DualCrossTAP model, retaining a substantial portion of its performance capabilities, albeit to a lesser extent than CNN-S1. An examination of per-activity classification reveals that CNN-S2 maintained a consistent F1 Score across most activities, with the *play_phone* activity exhibiting the lowest performance. For ease of comprehensive comparative analysis, the performance metrics of CNN-S1 and CNN-S2 are detailed in Table 8.

Following a similar investigative approach as with the CNN-S1 model, various quantization methods were explored for the CNN-S2 architecture. The primary objective was to identify an optimal configuration that is less resource-intensive, thereby ensuring efficient deployment on MCUs. A detailed comparative analysis of these techniques, evaluating their impact on accuracy, F1 Score (macro), precision (macro), model size, and deployability, is presented in Table 9. For reasons previously articulated, namely limitations within the TFLM framework and the ESP32-S3 hardware, float-16 and mixed precision quantization methods could not be ultimately utilized. This was despite the promising performance of 16-bit floating-point quantization, which, as indicated in Table 9, achieved an F1 Score of 97.67 % and an accuracy of 97.66 %. These metrics are almost identical to those of the non-quantized model (97.68 % F1 Score and accuracy) while reducing the model size from 127.984 KB to 68.952 KB. Conversely, the 8-bit integer-quantized model is supported by TFLM and is deployable on the target hardware, offering a comparable model size of 50.856 KB similar to mixed precision. However, this method resulted in a significant degradation in performance, with the F1 Score dropping to 89.53 % and accuracy to 89.92 % from the 97.68 % achieved by the non-quantized model. This substantial decrease in classification performance outweighs the benefits of model size reduction and inference latency. Consequently, considering the trade-off between model size, computational resources, and classification efficacy, the non-quantized (vanilla) CNN-S2 model, with its superior F1 Score of 97.68 % and manageable size of 127.984 KB, was determined to be the most suitable choice for deployment on the ESP32-S3. Therefore, all subsequent experimentation and deployment procedures proceeded with the vanilla CNN-S2 model, and the quantized alternatives were discarded.

Upon deployment on the ESP32-S3 MCU, the CNN-S2 model performed as anticipated, achieving an accuracy of 97.68 % and an F1 Score (macro) of 97.68 %. The resource utilization metrics are detailed in Table 11. Specifically, the inference latency for the CNN-S2 model was measured at approximately 639,543 microseconds. This represents a significant improvement, being nearly three times lower than the 1,836,271-microsecond latency recorded for the CNN-S1 model. In terms of memory, the CNN-S2 model exhibited a peak SRAM consumption of 82 KB during inference and a Flash memory footprint of 127.984 KB. The increase in model SRAM consumption (82 KB for CNN-S2 compared to 62 KB for CNN-S1) is attributable to the architectural design employing *depthwise separable convolutional* layers, as discussed in Subsection 4.3. Unlike standard *convolutional* layers, which translate to a single *CONV_2D* operation in TFLM, depthwise separable convolutions correspond to two distinct operations (*DEPTHWISE_CONV_2D* and *CONV_2D*). This structural difference inherently requires more operational memory. However, this design choice yields considerable advantages, including a reduced model Flash size (127.984 KB for CNN-S2 versus 209.108 KB for CNN-S1) and the aforementioned substantial reduction in inference latency. The total system-wide SRAM consumption on the ESP32-S3 platform when running the CNN-S2 model was 111.676 KB. This comprised 82 KB for the CNN-S2 model itself, 10.8 KB for the input buffer, and the remainder allocated to system-level components such as FreeRTOS, UART drivers, and string literals. Similarly, the total Flash memory utilization for the system was 476.457 KB, of which the model accounted for 127.984 KB, with the remainder consumed by other system software and data. Consistent with the testing protocol for CNN-S1, the ESP32-S3 was configured to operate at a constant CPU frequency of 240 MHz, without DFS or other power-saving alterations. Under these conditions, the average power consumption during the entire testing period for CNN-S2 was measured at 215 mW. This represents a minor increase of approximately 5 mW compared to the CNN-S1 model (210 mW). We suspect that this increased power draw is a consequence of the higher number of OPS executed by the *depthwise separable convolutional* layers and the associated rise in active memory utilization.

To extend battery life, we conducted further experiments to fine-tune the operational parameters of the CNN-S2 model deployed on the ESP32-S3, similar to CNN-S1. This involved evaluating performance across a range of fixed CPU frequencies (from 10 MHz to 240 MHz) and utilizing DFS. The primary objective was to identify an optimal balance between inference latency and power consumption. Concurrently, CPU temperatures were monitored using the ESP32-S3's internal sensor to assess whether external thermal management solutions would be necessary. The comprehensive results of this investigation are presented in Table 12. A general observation, consistent with initial deployment metrics, is that the CNN-S2 model tends to exhibit slightly

Table 10
Per-layer and per-operation execution time for CNN-S2 on ESP32-S3, running at a constant CPU frequency of 240 MHz. Execution times were measured as the average of 5000 randomly sampled inference runs, with detailed per-operation latency profiles collected during benchmarking. During the conversion from TensorFlow to TFLite format, the converter replaced all 1D separable convolutions with 2D separable convolutions, as native support for 1D convolution is currently unavailable in TFLM. Since 1D convolution is a special case of 2D convolution (Contributors, 2021; Developers, 2024), the TFLite converter introduces a *Reshape* layer to transform 1D data into 2D format, ensuring compatibility. Finally, the last row reports peak SRAM utilization of 82 KB and the total execution time (inference latency) of 639,543 micro seconds.

Layer	Operation	Operation Execution Time (µs)	Layer Total Execution Time (µs)
Reshape	SHAPE	13	99
	STRIDED_SLICE	36	
	PACK	14	
	RESHAPE	36	
Separable Convolution	DEPTHWISE_CONV_2D	7347	61768
	CONV_2D	54,421	
Max Pool	MAX_POOL_2D	6049	6049
Separable Convolution	DEPTHWISE_CONV_2D	12,883	188831
	CONV_2D	175,948	
Max Pool	MAX_POOL_2D	6012	6012
Separable Convolution	DEPTHWISE_CONV_2D	12,844	361670
	CONV_2D	348,826	
Global Average Pooling	MEAN	11,939	11,939
Dense	FULLY_CONNECTED	2679	2679
Output (Dense)	FULLY_CONNECTED	447	496
	SOFTMAX	49	
	82 KB (SRAM)		**639,543**

Table 11
Comparison of the deployed CNN-S1 and CNN-S2 student models on the ESP32-S3 MCU. The *Model SRAM* and *Model Flash* columns indicate the SRAM and Flash memory utilization specific to each model, while the *Total SRAM* and *Total Flash* columns represent the overall system-wide SRAM and Flash memory consumption on the ESP32-S3.

Student Model	F1 Score (macro)	Inference Latency	Model SRAM	Model Flash	Total SRAM	Total Flash	Power Draw
CNN-S1	98.36 %	1836271 µs	62 KB	209.108 KB	91.636 KB	552.649 KB	210 mW
CNN-S2	97.68 %	639543 µs	82 KB	127.984 KB	111.676 KB	476.457 KB	215 mW

Table 12
Performance evaluation of the ESP32-S3 using the CNN-S2 student model at various CPU frequencies. In *DFS* mode, the CPU dynamically operates between a minimum of 80 MHz and a maximum of 240 MHz. The displayed CPU temperatures were recorded using the ESP32-S3's built-in sensor.

Mode	CPU Frequency (MHz)	CNN-S1 Inference Latency (µs)	Power Consumption (mW)	CPU Temperature (K)
DFS	min: 80 and max: 240	639529	216	307.75
Fixed/Constant	240	639,543	215	307.7
	160	959,294	144	306.75
	80	1,933,541	119	305.75
	40	3,958,349	75	304.75
	20	8,304,935	66	303.75
	10	18,403,205	60	304.75

higher power consumption at equivalent CPU frequencies compared to CNN-S1. However, the overall patterns of how inference latency, power draw, and CPU temperature scale with frequency for CNN-S2 are strikingly similar to those observed for CNN-S1. Consequently, many of the same conclusions regarding frequency selection apply, albeit with the significant caveat that CNN-S2 is considerably faster across the board. For instance, CNN-S2 operating at a fixed 160 MHz achieves an inference latency of 959,294 µs. This is approximately 1.9 times faster than the CNN-S1 model operating at its peak performance at 240 MHz (which had a latency of 1,836,271 µs, as noted in prior comparisons), and CNN-S2 achieves this superior speed while consuming less power (144 mW at 160 MHz for CNN-S2 versus 210 mW at 240 MHz for CNN-S1). Based on the data in Table 12, the 160 MHz fixed frequency again emerges as a highly favorable operating point, offering an excellent compromise between performance and energy efficiency for CNN-S2. While it consumes more power (144 mW) and results in a slightly higher CPU temperature

Table 13
Comparison of our optimized CNN-S1 and CNN-S2 models against state-of-the-art approaches, considering only the IMU sensor. To compute *Total Parameters*, *Trainable Parameters*, and *MFLOPs* from the study (Luo et al., 2024), we referred to their cited GitHub repository. As their models are implemented in PyTorch, we employed *ptflops* (Contributors, 2024a) and *summary* (Contributors, 2024b) for MFLOPs and model summary calculations.

Study	Model	Total Parameters	Trainable Parameters	MFLOPs	Compute	F1 Score
Luo et al. (2024)	DeepConvLSTM	558,293	558,293	606.03	N/A	82.8 %
	FCN	192,213	192,213	17.16		98.1 %
	DCNN	22,530,495	22,530,495	507.14		96.6 %
	TPN	91,061	91,061	46.47		88.6 %
	Shallow-LSTMs	426,197	426,197	531.80		79.1 %
	CNN-TA	731,466	731,466	66.96		97.8 %
This Study	**CNN-S1**	**51,701**	**51,701**	**6.55**	**ESP32-S3**	**98.36 %**
	CNN-S2	**30,288**	**30,288**	**2.19**		**97.68 %**

Table 14
Comprehensive comparison between our optimized models and recent state-of-the-art approaches reported in the literature. The evaluation focuses on model complexity through key metrics such as the total number of parameters, trainable parameters, and MFLOPs, while the *Utilized PP* column indicates whether a PP sensor was incorporated. Due to the differences in datasets used across studies, a fair comparison was achieved by reimplementing the external models with our dataset's input dimensions and output configuration (i.e., number of classification classes). Importantly, no changes were made to the intrinsic model architectures; only the input shape, final Dense layer output neurons, and any consequent adjustments were modified. A detailed discussion of the reimplementation process and metric calculations is provided in Section A.

Study	Model	Total Parameters	Trainable Parameters	MFLOPs	Utilized PP
Abdellatef et al. (2025)	HARCNN	219,763	219,763	9.67	✗
AlMuhaideb et al. (2024)	ResLSTM	856,107	853,227	107.02	✗
Park et al. (2025)	HT-AggNet	2,550,293	2,550,293	17.09	✗
Kumar and Suresh (2024)	Static Context CNN	19,666,197	19,666,197	53.07	✗
	Dynamic Context RNN	27,285	27,285	10.62	✗
	Dynamic Context LSTM	12,771	12,771	7.35	✗
This Study	**DualCrossTAP**	**690,711**	**688,279**	**67.90**	✓
	CNN-S1	**51,029**	**51,029**	**6.14**	✓
	CNN-S2	**30,288**	**30,288**	**2.19**	✓

(306.75 K) compared to the 80 MHz setting (119 mW, 305.75 K), the nearly halved inference latency (959,294 µs vs. 1,933,541 µs) justifies this trade-off. The DFS mode, operating between 80 MHz and 240 MHz, delivered performance (639,529 µs latency, 216 mW power) closely mirroring the fixed 240 MHz setting. Across all tested frequencies, the CPU temperatures for CNN-S2 remained modest, ranging from approximately 303.75 K to 307.75 K, indicating that active external cooling is not required for the ESP32-S3 under these workloads. The conclusion is also true for CNN-S1, which depicts a similar temperature range (see Subsection 5.2). Lastly, in Table 13 we compare our optimized and deployed CNN-S1 and CNN-S2 models against existing state-of-the-art models that are trained with CAPPIMU dataset which validate the effectiveness of our employed KD fusion (PP to IMU) strategy. To further validate our models' superior optimization and performance balance, Table 14 provides a comprehensive comparative analysis of recent state-of-the-art in the HAR domain alongside our trained and optimized models.

6. End to end prototype development and analysis

While previous sections focused on model evaluation and performance metrics in controlled settings, such analyses may not fully capture the complexities of real-world operational environments. Therefore, to validate the practical viability of our approach and to gain a comprehensive understanding of the engineering aspects involved, we developed a

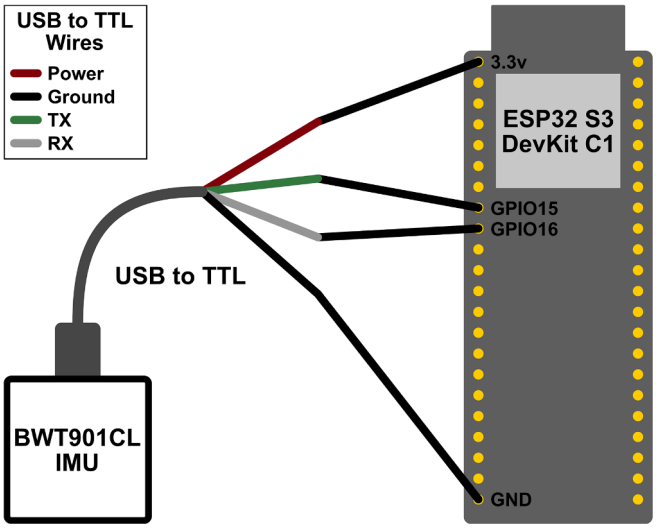


Fig. 3. High-level representation of the hardware configuration for the end-to-end prediction prototype system, featuring the ESP32-S3 Dev Kit C1 and the WitMotion BWT901CL IMU sensor. The WitMotion BWT901CL IMU sensor is powered by a 3.3 V supply from the ESP32-S3 Dev Kit C1, while the ESP32-S3 Dev Kit C1 itself is powered via USB connection to the workstation.

complete end-to-end prototype. This endeavor provides critical insights into the entire process, from sensor data acquisition to model inference, benefiting practitioners, engineers, and manufacturers in the development of actual low-cost HAR products.

The hardware configuration for this end-to-end system is foundational. Given our work with IMU data, and to ensure consistency with the CAPPIMU dataset, we selected the precise IMU sensor employed by the dataset's authors. The choice of sensor is crucial, as parameters such as scale factors, internal filters, and axis orientations can vary significantly between different IMU models. Utilizing an alternative sensor would necessitate considerable recalibration and data transformation efforts to align its output with the characteristics of the original dataset, potentially introducing inconsistencies. Consequently, for reasons of fidelity and reproducibility, we employed the WitMotion BWT901CL (Wit-Motion, 2025) IMU sensor. This sensor was interfaced with an ESP32-S3 Dev Kit C1 microcontroller via a USB to Transistor-Transistor Logic (TTL) converter. The overall hardware setup is depicted in Fig. 3. The BWT901CL IMU sensor is powered by a 3.3 V pin from the ESP32-S3, with its ground pin connected to the ESP32-S3's GND pin. The BWT901CL's transmit (TX) and receive (RX) pins are connected to General-Purpose Input/Output (GPIO) pins 16 and 15, respectively, on the ESP32-S3 board, facilitating UART communication.

Before deploying the machine learning models, it is essential to delineate the operational workflow, from raw data collection to the final inference output. The high-level steps are as follows –

I. **Data Acquisition from IMU Sensor**: The initial step involves collecting a single window of sensor data. In our configuration, this corresponds to 300 timestamps, captured at a sampling rate of 50 Hz over a six-second interval.
II. **Data Preprocessing**: Subsequently, the acquired window of raw sensor data undergoes a two-stage preprocessing. First, the raw 16-bit integer values are converted into their respective physical units using sensor-specific scale factors. Second, the data is normalized using z-score normalization, as detailed in Section 3 and Eq. (5), aligning with the methodology used for the CAPPIMU dataset.
III. **Model Inference**: Finally, the preprocessed data window is fed into the deployed TFLM model on the ESP32-S3 to obtain the activity prediction. This cycle of data collection, preprocessing, and inference is then repeated for continuous operation.

While the inference phase has been discussed previously, the specifics of data acquisition and preprocessing in the deployment context warrant further detail. To streamline data collection from the BWT901CL IMU sensor, we utilized the generic C library provided by WITMOTION (2024), which is compatible with C/C++ frameworks, including the Arduino framework used for the ESP32-S3. This library simplifies the process of reading sensor data via UART by handling low-level communication protocols and providing a callback mechanism via its API that is invoked when new data is available.

The data received from the sensor via this library is in a 16-bit integer (short) format. These raw integer values for accelerometer, gyroscope, and magnetometer readings must first be converted to meaningful physical units (e.g., m/s^2 for acceleration, $^\circ/s$ for angular velocity). This conversion is achieved by applying specific scale factors, as derived from the sensor's datasheet (Ltd., 2025). For instance, the accelerometer data is scaled using a factor of $(16.0 \times 9.8)/32768.0$, and gyroscope data by $2000.0/32768.0$. After this conversion, the sensor data for each of the nine axes (three for accelerometer, three for gyroscope, and three for magnetometer) is normalized. This normalization employs the z-score method (see Eq. (5)), utilizing the per-axis mean (μ) and standard deviation (σ) values (a total of 18 parameters) pre-calculated from the CAPPIMU dataset following dataset's authors. Once these preprocessing steps are completed, the data is in the appropriate format and scale for input to the deployed inference model.

$$z = \frac{x - \mu}{\sigma} \tag{5}$$

For deployment and evaluation on the end-to-end prototype system, both the CNN-S1 and CNN-S2 student models were selected. This choice was guided by their distinct characteristics, as previously detailed in Subsection 4.3: CNN-S1 is optimized for higher accuracy, whereas CNN-S2 is designed for superior inference latency. To comprehensively assess system performance under varying computational loads, both models were tested on the ESP32-S3 MCU operating at two distinct CPU frequencies: 240 MHz and 160 MHz. This dual-frequency approach allows for an analysis of the trade-offs between processing speed and power consumption, particularly relevant given the inclusion of the IMU sensor as an additional power-consuming component. Although prior investigations indicated 160 MHz as an optimal frequency for balancing performance and power efficiency, results for 240 MHz are also presented. The higher frequency, despite its increased power draw, offers reduced inference latency, and documenting its performance provides valuable data for practitioners and engineers considering different operational priorities. The detailed performance metrics for these deployment configurations are presented in Table 15.

Analyzing the performance of the CNN-S1 model, as shown in Table 15, reveals that at an operating frequency of 240 MHz, the total time to process one six-second window of data including inference (sum of RDCT, FCT, NT, and IL) is $6,000,010\,\mu s + 73\,\mu s + 550\,\mu s + 1,844,941\,\mu s = 7,845,574\,\mu s$ (approximately 7.85 s). This processing duration is considered acceptable for composite human activity recognition tasks, which often inherently involve longer data windows and do not typically demand near-instantaneous response times (Chen & Nugent, 2019). The total system power consumption at this frequency, inclusive of the IMU sensor, was measured at 275 mW. When the ESP32-S3 operating frequency was reduced to 160 MHz, the total processing time for CNN-S1 increased to $6,000,015\,\mu s + 95\,\mu s + 1,025\,\mu s + 2,705,335\,\mu s = 8,706,470\,\mu s$ (approximately 8.71 s). In exchange for this increased latency, the power consumption decreased significantly to 195 mW. This duration remains well within practical limits for the target applications.

The total SRAM utilization for the CNN-S1 deployment is 92.22 KB. This comprises several components: the TFLM model requires approximately 62 KB; the input buffer for 300 timestamps (2700 float values) accounts for 2700×4 Bytes = 10.8 KB; the IMU sensor's internal register buffer (for raw 16-bit integer data) consumes about 288 Bytes; temporary storage for IMU feature conversion (scaled accelerometer, gyroscope, and magnetometer values) uses around 12 Bytes; and constants for IMU feature normalization (mean and standard deviation for nine axes) require $9 \times 2 \times 4$ Bytes = 72 Bytes. The remaining SRAM is allocated to FreeRTOS, system libraries (e.g., serial communication, WitMotion library), logging functions, and other temporary variables. The total flash memory footprint for this configuration is 545.749 KB, of which the CNN-S1 model itself occupies 209.108 KB. The remainder is used by the real time operating system, device drivers, libraries, and other system components.

The deployment of the CNN-S2 model, engineered for lower latency, demonstrated faster processing times as anticipated. At 240 MHz, the total processing time was $6,000,008\,\mu s + 77\,\mu s + 558\,\mu s + 642,313\,\mu s = 6,642,956\,\mu s$ (approximately 6.64 s). This configuration resulted in a power consumption of 295 mW. At 160 MHz, the CNN-S2 model processed a data window in $6,000,012\,\mu s + 93\,\mu s + 1,028\,\mu s + 959,294\,\mu s = 6,960,427\,\mu s$ (approximately 6.96 s), with a corresponding power draw of 205 mW. Compared to CNN-S1, CNN-S2 offers a substantial reduction in inference latency and overall processing time, albeit with a slight increase in power consumption at both frequencies and higher SRAM usage.

The total SRAM consumption for the CNN-S2 deployment is 112.26 KB. The breakdown includes approximately 82 KB for the model's TFLM arena, with the input buffer (10.8 KB), IMU raw data buffer (288 Bytes), feature conversion temporaries (12 Bytes), and normalization constants (72 Bytes) remaining consistent with the CNN-S1 setup. The increased total SRAM is primarily due to the larger arena size

Table 15
Performance metrics collected from the end-to-end prototype deploying the student models CNN-S1 and CNN-S2 on the ESP32-S3 at operating frequencies of 240 MHz and 160 MHz. The *Frequency* column indicates the constant operating frequency for each model. *RDCT* (Raw Data Collection Time) denotes the duration required to capture one window of data utilizing BWT901CL IMU sensor, while *FCT* (Feature Conversion Time) represents the time taken to convert the raw data into physical units using sensor-specific scale factors. *NT* (Normalization Time) indicates the time required to normalize the data window, and *IL* (Inference Latency) measures the duration from feeding the normalized input to obtaining the model's prediction. Finally, *TS* and *TF* refer to the total allocations of SRAM and Flash memory, respectively, and *PD* reports the corresponding average power draw.

Model	Frequency	RDCT	FCT	NT	IL	TS	TF	PD
CNN-S1	240 MHz	6000010 µs	73 µs	550 µs	1844941 µs	92.22 KB	545.749 KB	275 mW
	160 MHz	6000015 µs	95 µs	1025 µs	2705335 µs			195 mW
CNN-S2	240 MHz	6000008 µs	77 µs	558 µs	642313 µs	112.26 KB	469.573 KB	295 mW
	160 MHz	6000012 µs	93 µs	1028 µs	959294 µs			205 mW

required by the CNN-S2 model. The rest of the memory is utilized by the system software and libraries. The flash memory usage for this model is 469.573 KB, with the CNN-S2 model binary contributing 127.984 KB.

The empirical results demonstrate that both the CNN-S1 and CNN-S2 models, when deployed on the ESP32-S3, present viable options for real-world applications. The choice between them, and the selection of operating frequency, will inevitably depend on the specific constraints and priorities of the target use case, such as the relative importance of predictive accuracy, inference speed, and power efficiency. While 240 MHz offers faster processing, the 160 MHz frequency generally provides a more favorable balance between performance and power consumption for both models, significantly reducing energy demand with a manageable increase in latency. These findings underscore the practical feasibility of deploying sophisticated deep learning models like CNN-S1 and CNN-S2 on resource-constrained embedded systems. Notably, this is achieved while maintaining parameter efficiency that is orders of magnitude superior to the original, more complex teacher DualCrossTAP model. As indicated in Table 15, even with the overhead of data acquisition and preprocessing, the system achieves inference latencies practical for many human activity recognition scenarios. Furthermore, the memory footprints indicate that sufficient resources remain on the ESP32-S3 for other concurrent tasks or more complex application logic.

Considering battery-powered applications, particularly those aiming for extended operational longevity using compact power sources such as coin cells (e.g., CR2032 Energizer, 2024, often operating around 3.0–3.3 V with capacities in the range of 220–240 mAh Leafony, 2025), the measured active power consumption figures (195 mW to 295 mW) would indeed lead to rapid battery depletion if the system were maintained in a continuously active state. For instance, a sustained draw of 195 mW at 3.3 V supply (ESP32-S3 requires 3.3 V) translates to an approximate current of 59 mA. This level of current would exhaust a 220 mAh coin cell in roughly $220\,\text{mAh}/59\,\text{mA} \approx 3.72$ h. However, it is crucial to contextualize these power measurements as likely representing a conservative, near worst-case scenario for a field-deployed product. This assessment is based on several factors. Firstly, as detailed in Subsection 5.2, the presented power consumption values pertain to the entire ESP32-S3 Dev Kit C1 evaluation board. In a realistic product designed for power efficiency, a custom-designed Printed Circuit Board (PCB) would be developed. Such PCB would be optimized for the specific application, omitting superfluous components typically found on development kits. These include, but are not limited to, USB ports and their associated bridge circuitry (e.g., USB to UART converters), unutilized GPIO pins, joint test action group debugging interfaces, and potentially less efficient power regulation components. The exclusion of such non-essential circuitry directly contributes to lower overall quiescent and active power consumption (Systems, 2025a), as well as reduced heat dissipation and a smaller physical dimensions. Secondly, significant power savings could be realized through a more sophisticated execution pipeline and dynamic power management strategies. In the current prototype, the ESP32-S3 was operated at a constant frequency (either 240 MHz or 160 MHz) throughout the entire process. A more power-aware approach could involve manual frequency setting, where the microcontroller operates at a substantially lower frequency (e.g., 20 MHz) during less computationally intensive phases, such as the raw data collection from the IMU sensor. The frequency could then be intelligently ramped up to higher levels (e.g., 160 MHz or 240 MHz) only when necessary for demanding tasks like feature conversion, normalization, and model inference. Furthermore, during the data collection period, the ESP32-S3 could leverage more advanced low-power states, such as the light sleep mode (Systems, 2025b). In light sleep, the CPU is paused, but peripherals and memory can remain active, allowing data acquisition to continue with minimal CPU power draw, thereby further reducing the average energy consumption per cycle. The implementation of these advanced power optimization techniques was beyond the primary scope of this study, which centered on establishing a complete end-to-end functional prototype for the HAR system. Nevertheless, this discussion is included with the intention of providing valuable perspectives for practitioners and engineers focused on developing production-grade, power-efficient embedded systems for composite human activity recognition. The principles outlined highlight feasible pathways to substantially extend battery life in practical deployments.

7. Limitation and future work

Despite the promising results achieved with the proposed KD-based sensor fusion strategy, the developed student models (CNN-S1 and CNN-S2), and the end-to-end prototype, this research is in its foundational stages and presents numerous avenues for further investigation and enhancement. A primary limitation of this study is its reliance on a single dataset, the CAPPIMU dataset. This was necessitated by the fact that CAPPIMU is currently the only publicly accessible dataset that incorporates synchronized data from both PP and IMU sensors. Future research should prioritize the development and public dissemination of additional datasets featuring PP sensor data. Such efforts would not only provide a richer empirical basis for understanding plantar pressure dynamics in human activity recognition but also address the current under-exploration of PP sensor modality within this domain. Furthermore, it is highly recommended that new datasets include raw sensor data in its original integer format alongside the pre-processed data converted into physical quantities. This approach offers distinct advantages for deployment on resource-constrained devices: it obviates the need for the feature conversion step (FCT), thereby conserving CPU cycles (as evidenced by metrics in Table 15). Moreover, working directly with integer data could potentially reduce the necessity for explicit normal-

ization if the raw data from different sensors or axes inherently share comparable scales. Crucially, it would also streamline the application of quantization-aware training (Contoli & Lattanzi, 2023; Kirtas et al., 2022), facilitating the development of fully integer-based models that can operate efficiently without dedicated floating-point units, leading to potential cost and power savings in hardware.

Another area for expansion pertains to a more exhaustive exploration of PP sensor data. While this study utilized PP data and investigated IMU-to-PP sensor fusion, the full potential of PP sensors, either as a standalone modality or in more diverse fusion configurations, remains largely untapped due to its nascent status in the HAR field. Building upon the sensor fusion strategies explored, future work could investigate an even more aggressive knowledge distillation approach. Specifically, a teacher model could be trained using comprehensive data from PP and IMU with the objective of distilling this multi-faceted knowledge into a highly compact student model that relies solely on accelerometer data. Such a strategy could lead to exceptionally resource-efficient student models without entirely sacrificing the rich contextual information provided by the other sensors.

The generalizability of the proposed teacher model, DualCrossTAP, also warrants further investigation. Its validation was confined to the CAPPIMU dataset. Therefore, future studies should endeavor to evaluate the performance of DualCrossTAP and the associated distillation methodologies on other benchmark HAR datasets, particularly those involving multi-sensor fusion, to ascertain its robustness and broader applicability. Additionally, for the student models CNN-S1 and CNN-S2, we examined various post-training quantization methods for optimization. However, we did not explore quantization-aware training or pruning (Cheng et al., 2024), which could have significantly optimized the parameters of CNN-S1. Future research should address these aspects or build upon them to further enhance model efficiency and performance.

In terms of model architecture, while this work extensively explored lightweight CNN designs, other efficient architectures, such as those based on dense blocks or alternative lightweight paradigms, were not investigated. An intriguing direction for future research is the application of knowledge distillation from neural networks to simpler, yet highly efficient, tree-based models, such as decision trees or random forest (Frosst & Hinton, 2017). Lightweight tree-based models are recognized for their exceptional inference speed and minimal resource consumption (Aytekin, 2022; Rafee et al., 2025). Distilling complex neural network knowledge into such models could further drastically reduce computational demands, enabling deployment on even more severely resource-constrained MCUs.

Finally, an analysis of the per-layer and per-operation execution times for CNN-S1 and CNN-S2 (detailed in Tables 6 and 10) consistently reveals that the *CONV_2D* operation constitutes the most significant computational bottleneck during inference across all tested ESP32-S3 frequencies. Consequently, a substantial improvement in inference latency could be achieved by designing and integrating a low-cost, low-compute hardware NN accelerator specifically for *CONV_2D* operations (Ahmadifarsani et al., 2024; Huang et al., 2025). While developing custom hardware accelerators is a challenging endeavor, frameworks and tools such as Apache TVM (Foundation, 2025) and the CFU Playground (Prakash et al., 2023) can aid in this process by facilitating the design, simulation, and deployment of such accelerators.

We aim to address these limitations and explore these promising research directions in our future work, with the overarching goal of further refining our sensor fusion and model compression techniques and expanding their practical applicability to a wider array of real-world embedded systems.

8. Discussion on broader impact and AI accessibility

The rapid advancement of AI, exemplified by models like AlphaFold (Guo et al., 2024b), promises transformation but also risks deepening the global resource divide by demanding immense computational power (Guo et al., 2025b; Wang et al., 2025a). Our work directly confronts this critical issue by proposing an innovative composite HAR method tailored for resource-constrained IoT devices. We introduce a new paradigm for the practical deployment of edge AI in wearable devices, achieving a breakthrough reduction in cloud dependency. Through hardware-aware optimization and the first systematic exploration of PP sensors within a KD framework, our approach successfully maintains high accuracy under the strict memory (less than 93 KB SRAM) and computational power constraints of the ESP32-S3. By compressing complex, cross-modal features into a miniature student model capable of real-time operation, we offer a feasible embedded AI solution for real-time-sensitive scenarios like remote medical monitoring. This not only makes advanced AI more accessible but also provides a scalable blueprint to mitigate the technological divide (Castro et al., 2024; Cupać et al., 2024), ensuring the benefits of the AI revolution are distributed more inclusively across the globe.

9. Conclusion

This research has successfully demonstrated a comprehensive approach to developing and deploying resource-efficient Human Activity Recognition systems by leveraging multi-modal data from Plantar Pressure and Inertial Measurement Unit sensors. Through the introduction of the DualCrossTAP teacher model and an effective knowledge distillation strategy, we developed compact yet capable student models (CNN-S1 and CNN-S2). The subsequent deployment and rigorous evaluation of these models on an end-to-end prototype confirmed their practical viability, achieving a crucial balance between predictive performance, inference latency, memory footprint, and power consumption suitable for resource-constrained embedded applications. This work underscores the potential of KD to enable sophisticated sensor fusion and intelligent decision-making on low-cost hardware.

While this study marks significant progress, it also opens avenues for future enhancements. Key directions include the development of more diverse, raw-integer-based PP and IMU synchronized datasets, the exploration of even more aggressive distillation strategies, and the potential for hardware acceleration of critical computational operations. Addressing these areas will further propel the deployment of robust and efficient embedded HAR solutions across a wider spectrum of real-world scenarios, from personalized healthcare to pervasive smart environments.

CRediT authorship contribution statement

Athar Noor Mohammad Rafee: Conceptualization, Methodology, Software, Validation, Formal analysis, Data curation, Investigation, Writing – original draft, Writing – review & editing, Funding acquisition; **John Clear:** Validation, Formal analysis, Investigation, Data curation, Writing – review & editing, Visualization, Funding acquisition; **Jannatun Noor:** Resources, Writing – review & editing, Supervision, Project administration.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Architectures of the reimplemented models for comparative study

In this section, we describe the architecture and reimplementation process for each study incorporated in our comparative analysis. Generally, the core architecture of each model remains unchanged; instead,

adjustments were made to the input shape, the number of classification classes, and other minor parameters to accommodate changes in input and output specifications. For studies that provided a complete codebase for model training, testing, and evaluation, we directly utilized that, and when necessary, extended the provided implementations to compute metrics such as *Total Parameters*, *Trainable Parameters*, and *MFLOPs*. In instances where the codebase was not provided, we adhered to the methodologies detailed in the respective papers by reimplementing the proposed architectures in TensorFlow and employing the *summary* and *profiler* APIs to calculate the required metrics.

When the codebase was provided, we employed the corresponding framework's APIs to compute the metrics. For example, for PyTorch implementations, we made use of the *summary* (Contributors, 2024b) and *ptflops* (Contributors, 2024a) libraries. The subsequent subsections offer a comprehensive discussion of each model's architecture and detail the reimplementation strategies employed for an effective comparison with our study.

A.1. HARCNN

We implemented the HARCNN model (Abdellatef et al., 2025) in TensorFlow by strictly adhering to the architecture presented in the original paper. The only modifications made were to the input configuration and the final Dense output layer. Further details regarding this implementation are provided in Table A.1.

A.2. ResLSTM

For the ResLSTM model (AlMuhaideb et al., 2024), we extended the original TensorFlow implementation provided by the authors by incorporating our custom input and output configurations. In addition, we made the necessary adjustments for calculating both the total number of parameters and the MFLOPs (see Table A.2 for details).

A.3. HT-AggNet

Similarly, for the HT-AggNet (Park et al., 2025) model, we extended the original PyTorch implementation provided authors (see Table A.3) by incorporating our custom input and output configurations.

A.4. Deep context models

Similar to the HARCNN model discussed previously in Subsection A.1, we implemented the proposed models in TensorFlow by closely following the methodologies outlined in the original research papers. Detailed information regarding the architectures and parameter configurations is presented in Tables A.4–A.6.

Table A.1
Modified HARCNN model architecture and parameters. The input shape is (300, 9) representing IMU sensor data with 9 features and 300 time steps. "N" denotes the batch size.

Stage	Layer Type	Parameters	Output Shape (N, …)
IMU Input	Input	Shape: (300, 9)	(N, 300, 9)
Conv Block 1	Conv1D	Filters: 32, Kernel: 5, Stride: 2, Padding: Same	(N, 150, 32)
	ReLU	–	
Conv Block 2	Conv1D	Filters: 48, Kernel: 3, Stride: 2, Padding: Same	(N, 75, 48)
	ReLU	–	
Concat Block 1	Conv1D (Branch A)	Filters: 48, Kernel: 3, Stride: 1, Padding: Same	(N, 75, 48)
	ReLU	–	
	Conv1D (Branch B)	Filters: 48, Kernel: 1, Stride: 1, Padding: Same	(N, 75, 48)
	ReLU	–	
	Concatenate	Axis: −1	(N, 75, 96)
	MaxPooling1D	Pool Size: 2, Strides: 2, Padding: Same	(N, 38, 96)
Conv Block 3	Conv1D	Filters: 64, Kernel: 3, Stride: 1, Padding: Same	(N, 38, 64)
	ReLU	–	
Concat Block 2	Conv1D (Branch A)	Filters: 64, Kernel: 3, Stride: 1, Padding: Same	(N, 38, 64)
	ReLU	–	
	Conv1D (Branch B)	Filters: 64, Kernel: 1, Stride: 1, Padding: Same	(N, 38, 64)
	ReLU	–	
	Concatenate	Axis: −1	(N, 38, 128)
	MaxPooling1D	Pool Size: 2, Strides: 2, Padding: Same	(N, 19, 128)
Conv Block 4	Conv1D	Filters: 128, Kernel: 3, Stride: 1, Padding: Same	(N, 19, 128)
	ReLU	–	
Concat Block 3	Conv1D (Branch A)	Filters: 128, Kernel: 3, Stride: 1, Padding: Same	(N, 19, 128)
	ReLU	–	
	Conv1D (Branch B)	Filters: 128, Kernel: 1, Stride: 1, Padding: Same	(N, 19, 128)
	ReLU	–	
	Concatenate	Axis: −1	(N, 19, 256)
	MaxPooling1D	Pool Size: 2, Strides: 2, Padding: Same	(N, 10, 256)
Feature Flattening	Flatten	–	(N, 2560)
Dense Block	Dense	Units: 21	(N, 21)
	ReLU	–	
Output	Dense	Units: 21 (num_classes)	(N, 21)
	Softmax	–	

Table A.2
ResLSTM architecture and parameters. The model takes three parallel input streams, each with shape (300, 3) representing sensor data with 3 features (3-axis accelerometer, 3-axis gyroscope, 3-axis magnetometer) and 300 time steps. Here "N" denotes the batch size.

Stage	Layer Type	Parameters	Output Shape (N, …)
Input Stream 1	Input	Shape: (300, 3)	(N, 300, 3)
Input Stream 2	Input	Shape: (300, 3)	(N, 300, 3)
Input Stream 3	Input	Shape: (300, 3)	(N, 300, 3)
Shared Processing Branch: CNN_ResBLSTM_Attention Sub-module (Applied to each input stream. Input to this branch is (N, 300, 3), output is (N, 128))			
CNN Block 1	Conv1D	Filters: 256, Kernel: 3, Padding: Same, Act: Swish	(N, 300, 256)
	BatchNormalization	–	(N, 300, 256)
	MaxPooling1D	Pool Size: 2, Strides: 2, Padding: Valid	(N, 150, 256)
	Dropout	Rate: 0.2	(N, 150, 256)
CNN Block 2	Conv1D	Filters: 128, Kernel: 3, Padding: Same, Act: Swish	(N, 150, 128)
	BatchNormalization	–	(N, 150, 128)
	MaxPooling1D	Pool Size: 2, Strides: 2, Padding: Valid	(N, 75, 128)
	Dropout	Rate: 0.2	(N, 75, 128)
CNN Block 3	Conv1D	Filters: 64, Kernel: 3, Padding: Same, Act: Swish	(N, 75, 64)
	BatchNormalization	–	(N, 75, 64)
	MaxPooling1D	Pool Size: 2, Strides: 2, Padding: Valid	(N, 37, 64)
	Dropout	Rate: 0.2	(N, 37, 64)
CNN Block 4	Conv1D	Filters: 32, Kernel: 3, Padding: Same, Act: Swish	(N, 37, 32)
	BatchNormalization	–	(N, 37, 32)
	MaxPooling1D	Pool Size: 2, Strides: 2, Padding: Valid	(N, 18, 32)
	Dropout	Rate: 0.2	(N, 18, 32)
ResBLSTM Block	LayerNormalization	–	(N, 18, 32)
	Bidirectional LSTM (x)	Units: 64, Return Sequences: True	(N, 18, 128)
	LayerNormalization	–	(N, 18, 128)
	Bidirectional LSTM (y)	Units: 64, Return Sequences: True	(N, 18, 128)
	Add	Inputs: [x, y]	(N, 18, 128)
Attention Mechanism	Attention (Custom)	Step Dim: 18	(N, 128)
Fusion and Output Head			
Feature Concatenation	Concatenate	From 3 branches, Input per branch: (N, 128)	(N, 384)
Classifier	Dropout	Rate: 0.3	(N, 384)
	Dense	Units: 21	(N, 21)
	Activation	Type: Softmax	(N, 21)

Table A.3
The architecture and parameters of the HT-AggNet model are outlined below. The input data, consisting of IMU sensor measurements, has a shape of $(N, 9, 300)$, where 9 represents the number of features, 300 denotes the time steps, and "N" is the batch size. The model is constructed with $L = 8$ TAggBlocks, utilizes an internal channel dimension of $nc_o = 128$, and is modified to perform classification across 21 classes.

Stage	Layer Type	Parameters	Output Shape (N, …)
IMU Input	Input	Shape: (9, 300) (Features, Timesteps)	(N, 9, 300)
Stem Convolution	Conv1D	In: 9, Out: 128, Kernel: 5, Padding: Same	(N, 128, 300)
Initial Pooling	MaxPool1d	Kernel: 3, Stride: 2, Padding: 1	(N, 128, 150)
TAggBlock Processing Loop (L = 8 iterations) (Input x to the first block is (N, 128, 150). The l_feat (N, 128, 150) from block $i-1$ is input x to block i.) (Each block i also outputs a g_feat (N, 16, 1) for final aggregation.)			
TAggBlock (generic)	Grouped Conv1D (*gconv*)	In: 128, Out: 128, Kernel: 5, Padding: Same, Groups: 8, Bias: False	(N, 128, 150)
	BatchNorm1d	Features: 128	(N, 128, 150)
	ReLU	In-place after BatchNorm	(N, 128, 150)
	ChannelShuffle	Groups: 8	(N, 128, 150) (l_feat)
	Temp. Global Conv1D (*gconv*)	In: 128, Out: 16 (128/L), Kernel: 150 (input T)	(N, 16, 1) (g_feat)
Output Head			
Feature Aggregation	Concatenate (*g_feats*)	L = 8 tensors of (N, 16, 1) along channel dim	(N, 128, 1)
Flattening	View	Target shape: (N, -1)	(N, 128)
Classifier	Fully Connected	In: 128, Out: 21	(N, 21)
	LogSoftmax	Dim: 1	(N, 21)

Table A.4
The architecture and parameters of the Static Context CNN model are detailed below. The input shape is $(300, 9, 1)$, which represents IMU sensor data reshaped to include a single channel, with 9 units as the width (features) and 300 time steps as the height. Here, "N" denotes the batch size.

Stage	Layer Type	Parameters	Output Shape (N, …)
IMU Input (reshaped)	Input	Shape: (300, 9, 1)	(N, 300, 9, 1)
Conv Block	Conv2D	Filters: 256, Kernel: (3,3), Padding: Same	(N, 300, 9, 256)
	ReLU	–	
	MaxPooling2D	Pool Size: (2,2)	(N, 150, 4, 256)
	Dropout	Rate: 0.25	(N, 150, 4, 256)
Feature Flattening	Flatten	–	(N, 153600)
Dense Block 1	Dense	Units: 128	(N, 128)
	ReLU	–	
	Dropout	Rate: 0.25	(N, 128)
Output Block	Dense	Units: 21 (num_classes)	(N, 21)
	Softmax	–	

Table A.5
The architecture and parameters of the Dynamic Context RNN model are outlined below. The input shape is $(300, 9)$, representing IMU sensor data captured over 300 time steps with 9 features, where "N" denotes the batch size. Additionally, the RNN layer was unrolled to ensure compatibility with TFLM.

Stage	Layer Type	Parameters	Output Shape (N, …)
IMU Input	Input	Shape: (300, 9) (Timesteps, Features)	(N, 300, 9)
Recurrent Layer	RNN (SimpleRNNCell)	Units: 128, Unroll: True	(N, 128)
Dropout Layer 1	Dropout	Rate: 0.2	(N, 128)
Dense Block 1	Dense	Units: 64	(N, 64)
	ReLU	–	
Dropout Layer 2	Dropout	Rate: 0.5	(N, 64)
Output Block	Dense	Units: 21 (num_classes)	(N, 21)
	Softmax	–	

Table A.6
The architecture and parameters of the Dynamic Context LSTM model are detailed below. The model accepts IMU sensor data with an input shape of $(300, 9)$, where 9 features are recorded over 300 time steps, and "N" denotes the batch size. Similar to the Dynamic Context RNN model, the LSTM layer was unrolled to ensure TFLM compatibility.

Stage	Layer Type	Parameters	Output Shape (N, …)
IMU Input	Input	Shape: (300, 9)	(N, 300, 9)
LSTM Layer 1	LSTM	Units: 30, Return Sequences: True, Unroll: True	(N, 300, 30)
LSTM Layer 2	LSTM	Units: 30, Unroll: True	(N, 30)
Output	Dense	Units: 21 (num_classes)	(N, 21)
	Softmax	–	

Appendix B. Teacher model architecture

In the below Table B.1 we present the detailed DualCrossTAP teacher model's architecture and parameters necessary to reproduce our study.

Table B.1
DualCrossTAP Teacher model architecture and parameters. Input shapes are based on IMU data with 9 features and PP data with 16 features, both sampled at 300 time steps. "N" denotes the batch size. Moreover, the optimal parameters are selected via an extensive grid search.

Stage	Layer Type	Parameters	Output Shape (N, …)
IMU Branch			
IMU Input	Input	Shape: (2700,)	(N, 2700)
	Reshape	Target Shape: (300, 9)	(N, 300, 9)
Conv Block 1	Conv1D	Filters: 32, Kernel: 3, Stride: 1, Padding: Same	
	Batch Norm	–	(N, 300, 32)
	ReLU	–	
	Residual Add	(Shortcut 1x1 Conv if needed)	
	MaxPooling1D	Pool Size: 2	(N, 150, 32)
Conv Block 2	Conv1D	Filters: 64, Kernel: 3, Stride: 1, Padding: Same	
	Batch Norm	–	(N, 150, 64)
	ReLU	–	
	Residual Add	(Shortcut 1x1 Conv if needed)	
	MaxPooling1D	Pool Size: 2	(N, 75, 64)
Conv Block 3	Conv1D	Filters: 128, Kernel: 3, Stride: 1, Padding: Same	
	Batch Norm	–	(N, 75, 128)
	ReLU	–	
	Residual Add	(Shortcut 1x1 Conv if needed)	
Transformer (IMU Features)	MultiHeadAttention	Heads: 4, Key Dim: 32	
	Dropout	Rate: 0.1	
	LayerNorm	Epsilon: 1e-6 (Residual Add)	(N, 75, 128)
	Dense (FFN)	Units: 128 (ReLU), Units: 128 (Input Dim)	
	Dropout	Rate: 0.1	
	LayerNorm	Epsilon: 1e-6 (Residual Add)	
Plantar Pressure Branch			
PP Input	Input	Shape: (4800,)	(N, 4800)
	Reshape	Target Shape: (300, 16)	(N, 300, 16)
Conv Block 1	Conv1D	Filters: 32, Kernel: 3, Stride: 1, Padding: Same	
	Batch Norm	–	(N, 300, 32)
	ReLU	–	
	Residual Add	(Shortcut 1x1 Conv if needed)	
	MaxPooling1D	Pool Size: 2	(N, 150, 32)
Conv Block 2	Conv1D	Filters: 64, Kernel: 3, Stride: 1, Padding: Same	
	Batch Norm	–	(N, 150, 64)
	ReLU	–	
	Residual Add	(Shortcut 1x1 Conv if needed)	
	MaxPooling1D	Pool Size: 2	(N, 75, 64)
Conv Block 3	Conv1D	Filters: 128, Kernel: 3, Stride: 1, Padding: Same	
	Batch Norm	–	(N, 75, 128)
	ReLU	–	
	Residual Add	(Shortcut 1x1 Conv if needed)	
Transformer (PP Features)	MultiHeadAttention	Heads: 4, Key Dim: 32	
	Dropout	Rate: 0.1	
	LayerNorm	Epsilon: 1e-6 (Residual Add)	(N, 75, 128)
	Dense (FFN)	Units: 128 (ReLU), Units: 128 (Input Dim)	
	Dropout	Rate: 0.1	
	LayerNorm	Epsilon: 1e-6 (Residual Add)	
Cross-Modal Interaction			
IMU Attends PP	MultiHeadAttention	Heads: 4, Key Dim: 32 (Query: IMU, Value: PP)	
	Add	(IMU Features + IMU Attended)	(N, 75, 128)
PP Attends IMU	MultiHeadAttention	Heads: 4, Key Dim: 32 (Query: PP, Value: IMU)	
	Add	(PP Features + PP Attended)	
Temporal Pooling and Classification Head			
IMU Path	TemporalAttention	Dense Units: 1 (tanh), Softmax over time	(N, 128)
PP Path	TemporalAttention	Dense Units: 1 (tanh), Softmax over time	
Concatenate	Concatenate	(IMU Pooled, PP Pooled)	(N, 256)
Dense Block 1	Dense	Units: 512, Activation: Swish, Regularizer: L2	
	Batch Norm	–	(N, 512)
	Dropout	Rate: 0.6	
Dense Block 2	Dense	Units: 256, Activation: Swish, Regularizer: L2	
	Batch Norm	–	(N, 256)
	Dropout	Rate: 0.5	
Output	Dense	Units: 21 (num_classes), Activation: Softmax	(N, 21)

Table C.1
Student CNN-S1 model architecture and parameters. The input shape is (300, 9) representing IMU sensor data with 9 features with 300 time steps. "N" denotes the batch size.

Stage	Layer Type	Parameters	Output Shape (N, ...)
IMU Input	Input	Shape: (300, 9)	(N, 300, 9)
Conv Block 1	Conv1D	Filters: 32, Kernel: 3, Stride: 1, Padding: Same	(N, 300, 32)
	ReLU	–	
	MaxPooling1D	Pool Size: 2	(N, 150, 32)
Conv Block 2	Conv1D	Filters: 64, Kernel: 3, Stride: 1, Padding: Same	(N, 150, 64)
	ReLU	–	
	MaxPooling1D	Pool Size: 2	(N, 75, 64)
Conv Block 3	Conv1D	Filters: 128, Kernel: 3, Stride: 1, Padding: Same	(N, 75, 128)
	ReLU	–	
Feature Pooling	GlobalAveragePooling1D	–	(N, 128)
Dense Block	Dense	Units: 128, Activation: ReLU	(N, 128)
	Dropout	Rate: 0.5	
Output	Dense	Units: 21 (num_classes)	(N, 21)
	Softmax	–	

Table C.2
Student CNN-S2 model architecture and parameters. The input shape is (300, 9) representing IMU data with 9 features with 300 time steps. "N" denotes the batch size. This model employs separable convolutions for improved efficiency compared to the previous CNN-S1.

Stage	Layer Type	Parameters	Output Shape (N, ...)
IMU Input	Input	Shape: (300, 9)	(N, 300, 9)
Conv Block 1	SeparableConv1D	Filters: 32, Kernel: 3, Stride: 1, Padding: Same	(N, 300, 32)
	ReLU	–	
	MaxPooling1D	Pool Size: 2	(N, 150, 32)
Conv Block 2	SeparableConv1D	Filters: 64, Kernel: 3, Stride: 1, Padding: Same	(N, 150, 64)
	ReLU	–	
	MaxPooling1D	Pool Size: 2	(N, 75, 64)
Conv Block 3	SeparableConv1D	Filters: 128, Kernel: 3, Stride: 1, Padding: Same	(N, 75, 128)
	ReLU	–	
Feature Pooling	GlobalAveragePooling1D	–	(N, 128)
Dense Block	Dense	Units: 128, Activation: ReLU	(N, 128)
	Dropout	Rate: 0.5	
Output	Dense	Units: 21 (num_classes)	(N, 21)
	Softmax	–	

Appendix C. Student model architecture

In this section we provide the designed student CNN models' architecture and parameters in Tables C.1 and C.2 respectively. The primary difference between these two models is that CNN-S1 utilizes *Conv1D* whereas CNN-S2 utilizes *SeparableConv1D*. Other than that, the entire architecture is identical to one another.

References

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M.,... Zheng, X. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems. Software available from tensorflow.org https://www.tensorflow.org/.

Abdellatef, E., Al-Makhlasawy, R. M., & Shalaby, W. A. (2025). Detection of human activities using multi-layer convolutional neural network. *Scientific Reports*, *15*(1), 7004. https://doi.org/https://doi.org/10.1038/s41598-025-90307-6

Ahmadifarsani, S., Stahl, R., van Kempen, P., Mueller-Gritschneder, D., & Schlichtmann, U. (2024). Towards rapid exploration of heterogeneous tinyML systems using virtual platforms and TVM's UMA. In *Proceedings of the 2023 workshop on compilers, deployment, and tooling for edge AI* CODAI '23 (p. 6-10). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3615338.3618121

AlMuhaideb, S., AlAbdulkarim, L., AlShahrani, D. M., AlDhubaib, H., & AlSadoun, D. E. (2024). Achieving more with less: A lightweight deep learning solution for advanced human activity recognition (HAR). *Sensors*, *24*(16). https://doi.org/10.3390/s24165436

Arduino, (2024). Serial - arduino reference. Accessed on 2024-06-09 https://www.arduino.cc/reference/en/language/functions/communication/serial/.

Aytekin, C. (2022). Neural networks are decision trees. https://arxiv.org/abs/2210.05189.

Burnie, L., Chockalingam, N., Holder, A., Claypole, T., Kilduff, L., & Bezodis, N. (2024). Testing protocols and measurement techniques when using pressure sensors for sport and health applications: A comparative review. *The Foot*, *59*, 102094. https://doi.org/10.1016/j.foot.2024.102094

Cao, L., Huo, T., Li, S., Zhang, X., Chen, Y., Lin, G., Wu, F., Ling, Y., Zhou, Y., & Xie, Q. (2024). Cost optimization in edge computing: A survey. *Artificial Intelligence Review*, *57*(11), 312. https://doi.org/10.1007/s10462-024-10947-4

Castro, K. A., Siwady, J. A., Castillo, E., Alonzo, A., Cardona, M., & Perdomo, M. E. (2024). Artificial intelligence for all: Challenges and harnessing opportunities in AI democratization. In *2024 IEEE International conference on machine learning and applied network technologies (ICMLANT)* (pp. 143–148). https://doi.org/10.1109/ICMLANT63295.2024.00030

Chen, L., & Nugent, C. D. (2019). Composite activity recognition. In *Human activity recognition and behaviour analysis: For cyber-physical systems in smart environments*, pp. 151–181). Cham: Springer International Publishing. https://doi.org/10.1007/978-3-030-19408-6_7

Chen, W., Li, Y., Tian, Z., & Zhang, F. (2023). 2D and 3D object detection algorithms from images: A survey. *Array*, *19*, 100305. https://doi.org/10.1016/j.array.2023.100305

Cheng, H., Zhang, M., & Shi, J. Q. (2024). A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *46*(12), 10558–10578. https://doi.org/10.1109/TPAMI.2024.3447085

Chitty-Venkata, K. T., Mittal, S., Emani, M., Vishwanath, V., & Somani, A. K. (2023). A survey of techniques for optimizing transformer inference. *Journal of Systems Architecture*, *144*, 102990. https://doi.org/10.1016/j.sysarc.2023.102990

Contoli, C., & Lattanzi, E. (2023). A study on the application of tensorflow compression techniques to human activity recognition. *IEEE Access*, *11*, 48046–48058. https://doi.org/10.1109/ACCESS.2023.3276438

Contributors, p. (2024a). ptflops: A python package for calculating FLOPs in pytorch models. Accessed on: October 31, 2024 https://github.com/sovrasov/flops-counter.pytorch.

Contributors, T. (2021). Dilated and causal convolutions on microcontrollers. Accessed: 2024-10-19 https://github.com/tensorflow/tensorflow/issues/48567.

Contributors, t. (2024b). View model summaries in pytorch! Accessed on: October 31, 2024 https://github.com/TylerYep/torchinfo.

Cupać, J., Schopmans, H., & Tuncer-Ebetürk, İ. (2024). Democratization in the age of artificial intelligence: Introduction to the special issue. *Democratization*, *31*(5), 899–921. https://doi.org/10.1080/13510347.2024.2338852

Daghero, F., Burrello, A., Xie, C., Castellano, M., Gandolfi, L., Calimera, A., Macii, E., Poncino, M., & Pagliari, D. J. (2022a). Human activity recognition on microcontrollers with quantized and adaptive deep neural networks. ACM Transactions on Embedded Computing Systems, *21*(4). https://doi.org/10.1145/3542819

Daghero, F., Jahier Pagliari, D., & Poncino, M. (2022b). Two-stage human activity recognition on microcontrollers with decision trees and CNNs. https://arxiv.org/abs/2206.07652.

De Guzman, S., Lowe, A., Williams, C., Kalra, A., & Anand, G. (2022). The development of a built-in shoe plantar pressure measurement system for children. *Sensors*, *22*(21). https://doi.org/10.3390/s22218327

Dehghani, A., Sarbishei, O., Glatard, T., & Shihab, E. (2019). A quantitative comparison of overlapping and non-overlapping sliding windows for human activity recognition using inertial sensors. *Sensors*, *19*(22), 5026. https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6891351/.

Deng, S., Chen, J., Teng, D., Yang, C., Chen, D., Jia, T., & Wang, H. (2024). LHAR: Lightweight human activity recognition on knowledge distillation. *IEEE Journal of Biomedical and Health Informatics*, *28*(11), 6318–6328. https://doi.org/10.1109/JBHI.2023.3298932

Developers, T. (2024). Tensorflow lite converter API. Accessed: 2024-09-21 https://www.tensorflow.org/api_docs/python/tf/lite/TFLiteConverter.

ESP32-S3-devkitc-1. (2023). Accessed: 2024-09-29 https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/index.html.

Elhanashi, A., Dini, P., Saponara, S., & Zheng, Q. (2024). Advancements in tinyML: Applications, limitations, and impact on iot devices. *Electronics*, *13*(17). https://doi.org/10.3390/electronics13173562

Energizer, (2024). Energizer CR2032 datasheet. Accessed: 2024-05-28 https://data.energizer.com/pdfs/cr2032.pdf.

Foundation, A. S. (2025). Apache TVM: Open deep learning compiler stack. Accessed: 2024-12-28 https://github.com/apache/tvm.

Frosst, N., & Hinton, G. (2017). Distilling a neural network into a soft decision tree. https://arxiv.org/abs/1711.09784.

Gan, M.-G., & Zhang, Y. (2023). Temporal attention-pyramid pooling for temporal action detection. *IEEE Transactions on Multimedia*, *25*, 3799–3810. https://doi.org/10.1109/TMM.2022.3166025

Gonçalves, P. H. N., Bragança, H., & Souto, E. (2024). Efficient human activity recognition on wearable devices using knowledge distillation techniques. *Electronics*, *13*(18). https://doi.org/10.3390/electronics13183612

Guo, Q., He, Z., & Wang, Z. (2024a). Monthly climate prediction using deep convolutional neural network and long short-term memory. *Scientific Reports*, *14*(1), 17748. https://doi.org/10.1038/s41598-024-68906-6

Guo, Q., He, Z., & Wang, Z. (2025a). Assessing the effectiveness of long short-term memory and artificial neural network in predicting daily ozone concentrations in liaocheng city. *Scientific Reports*, *15*(1), 6798. https://doi.org/10.1038/s41598-025-91329-w

Guo, S.-B., Meng, Y., Lin, L., Zhou, Z.-Z., Li, H.-L., Tian, X.-P., & Huang, W.-J. (2024b). Artificial intelligence alphafold model for molecular biology and drug discovery: A machine-learning-driven informatics investigation. *Molecular Cancer*, *23*(1), 223. https://doi.org/10.1186/s12943-024-02140-6

Guo, S.-B., Shen, Y., Meng, Y., Zhou, Z.-Z., Li, H.-L., Cai, X.-Y., Huang, W.-J., & Tian, X.-P. (2025b). Surge in large language models exacerbates global regional healthcare inequalities. *Journal of Translational Medicine*, *23*(1), 706. https://doi.org/10.1186/s12967-025-06751-5

He, Z., & Guo, Q. (2024). Comparative analysis of multiple deep learning models for forecasting monthly ambient PM2.5 concentrations: A case study in Dezhou city, China. *Atmosphere*, *15*(12). https://doi.org/10.3390/atmos15121432

Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. https://arxiv.org/abs/1503.02531.

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. https://arxiv.org/abs/1704.04861.

Huang, Z., Zandberg, K., Schleiser, K., & Baccelli, E. (2025). RIOT-ML: toolkit for over-the-air secure updates and performance evaluation of tinyml models. *Annals of Telecommunications*, *80*, 283–297. https://doi.org/10.1007/s12243-024-01041-5

Inmoresentum (2025). Esp32tflmwrapper. https://registry.platformio.org/libraries/inmoresentum/ESP32TFLMWrapper. PlatformIO library for ESP32 TensorFlow Lite Micro wrapper.

Ismail, W. N., Alsalamah, H. A., Hassan, M. M., & Mohamed, E. (2023). AUTO-HAR: An adaptive human activity recognition framework using an automated CNN architecture design. *Heliyon*, *9*(2), e13636. https://doi.org/10.1016/j.heliyon.2023.e13636

JetBrains (2024). Clion: A cross-platform ide for c and c++ by jetbrains. Accessed on: August 16, 2024 https://www.jetbrains.com/clion/.

Kagiyama, M., & Okita, T. (2025). Knowledge distillation for reservoir-based classifier: Human activity recognition. https://arxiv.org/abs/2505.22985.

Kaur, H., Rani, V., & Kumar, M. (2024). Human activity recognition: A comprehensive review. *Expert Systems*, *41*(11), e13680. https://doi.org/10.1111/exsy.13680

Keweisi (2025). Keweisi KWS-MX18l digital display detector voltmeter. Accessed: 2025-05-14 https://www.amazon.com/KWS-MX18L-Digital-Display-Detector-Voltmeter/dp/B0CGTG7JNQ.

Khan, M. T. R., Ever, E., Eraslan, S., & Yesilada, Y. (2025). Human activity recognition using binary sensors: A systematic review. *Information Fusion*, *115*, 102731. https://doi.org/10.1016/j.inffus.2024.102731

Kirtas, M., Oikonomou, A., Passalis, N., Mourgias-Alexandris, G., Moralis-Pegios, M., Pleros, N., & Tefas, A. (2022). Quantization-aware training for low precision photonic neural networks. *Neural Networks*, *155*, 561–573. https://doi.org/10.1016/j.neunet.2022.09.015

Kormanyos, C. (2024). xxd: hex-dump-type utility. https://github.com/ckormanyos/xxd. Accessed: 2024-09-29.

Kumar, P., & Suresh, S. (2024). Deep context model (DCM): Dual context-attention aware model for recognizing the heterogeneous human activities using smartphone sensors. *Evolving Systems*, *15*(4), 1475–1486. https://doi.org/10.1007/s12530-024-09570-z

Labrador, M. A., & Yejas, O. D. L. (2013). Human activity recognition: Using wearable sensors and smartphones. Chapman & Hall/CRC Computer and Information Science Series (1st ed.). Chapman and Hall/CRC. https://doi.org/10.1201/b16098

Leafony (2025). Leafony AV01 CR2032 documentation. Accessed: 2024-05-28 https://docs.leafony.com/en/docs/leaf/power/av01/.

Leite, C. S., Mauranen, H., Zhanabatyrova, A., & Xiao, Y. (2024). Transformer-based approaches for sensor-based human activity recognition: Opportunities and challenges. https://arxiv.org/abs/2410.13605.

Li, B., Liu, Y., Li, W., Xu, S., Yang, X., & Sun, Y. (2018). Foot plantar pressure measurement system based on flexible force-sensitive sensor and its clinical application. In *2018 IEEE 3rd Advanced information technology, electronic and automation control conference (IAEAC)* (pp. 1998–2002). https://doi.org/10.1109/IAEAC.2018.8577945

Li, H., Anatoljevic Baryskievic, I., Antonovich Baryskievic, A., & Yurevich Tsviatkou, V. (2025). A novel integration of bodily-kinesthetic intelligence (BKI) and feature mining methodology: Applications in fall risk assessments. *IEEE Sensors Journal*, *25*(5), 8721–8736. https://doi.org/10.1109/JSEN.2025.3531296

Liu, W., Xiao, Y., Wang, X., & Deng, F. (2021). Plantar pressure detection system based on flexible hydrogel sensor array and WT-RF. *Sensors*, *21*(17). https://doi.org/10.3390/s21175964

Ltd., W. S. C. (2025). Fcc datasheet for bwt901cl. Accessed: 26-May-2024 https://m.media-amazon.com/images/I/81q25mmcAyL.pdf.

Luo, B., Qiu, Q., Zhu, T., & Liu, Z. (2024). CAPPIMU: A composite activities dataset for human activity recognition utilizing plantar pressure and IMU sensors. In Z. Shi, J. Torresen, & S. Yang (Eds.), *Intelligent information processing XII* (pp. 87–100). Cham: Springer Nature Switzerland.

Nano 33 BLE Sense Rev2. (2025). https://docs.arduino.cc/hardware/nano-33-ble-sense-rev2/. Accessed: 2025-05-10.

Nordic nRF5340. (2025). https://www.nordicsemi.com/Products/nRF5340. Accessed: 2025-05-10.

Mardanpour, M., Sepahvand, M., Abdali-Mohammadi, F., Nikouei, M., & Sarabi, H. (2023). Human activity recognition based on multiple inertial sensors through feature-based knowledge distillation paradigm. *Information Sciences*, *640*, 119073. https://doi.org/10.1016/j.ins.2023.119073

Mekruksavanich, S., & Jitpattanakul, A. (2022). FallNeXt: A deep residual model based on multi-branch aggregation for sensor-based fall detection. *ECTI Transactions on Computer and Information Technology (ECTI-CIT)*, *16*(4), 352–364. https://doi.org/10.37936/ecti-cit.2022164.248156

Moreau, P., Durand, D., Bosche, J., & Lefranc, M. (2024). Two-branch neural network using two data types for human activity recognition. *IEEE Sensors Journal*, *24*(2), 2216–2227. https://doi.org/10.1109/JSEN.2023.3332290

Moslemi, A., Briskina, A., Dang, Z., & Li, J. (2024). A survey on knowledge distillation: Recent advancements. *Machine Learning with Applications*, *18*, 100605. https://doi.org/10.1016/j.mlwa.2024.100605

Muniasamy, A., & Asiri, F. (2025). Advanced human activity recognition on wearables with multi-scale sparse attention. *Multimedia Tools and Applications*, . https://doi.org/10.1007/s11042-024-20475-6

Murshed, M. G. S., Murphy, C., Hou, D., Khan, N., Ananthanarayanan, G., & Hussain, F. (2021). Machine learning at the network edge: A survey. *ACM Computing Surveys*, *54*(8). https://doi.org/10.1145/3469029

Park, J., Kim, D.-W., & Lee, J. (2025). HT-AggNet: Hierarchical temporal aggregation network with near-zero-cost layer stacking for human activity recognition. *Engineering Applications of Artificial Intelligence*, *149*, 110465. https://doi.org/10.1016/j.engappai.2025.110465

Passalis, N., Tzelepi, M., & Tefas, A. (2022). Chapter 8 - knowledge distillation. In A. Iosifidis, & A. Tefas (Eds.), *Deep learning for robot perception and cognition* (pp. 165–186). Academic Press. https://doi.org/10.1016/B978-0-32-385787-1.00013-0

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2024). Standardscaler. https://scikit-learn.org/dev/modules/generated/sklearn.preprocessing.StandardScaler.html. Accessed: 2024-10-04.

PlatformIO (2024). PlatformIO: Professional collaborative platform for embedded development. Accessed on: August 16, 2024 https://platformio.org/.

PlatformIO (2025). Arduino Framework - PlatformIO Documentation. https://docs.platformio.org/en/latest/frameworks/arduino.html. Accessed: 2025-05-10.

Polo-Rodríguez, A., Escobedo, P., Martínez-Martí, F., Marcen-Cinca, N., Carvajal, M. A., Medina-Quero, J., & Martínez-García, M. S. (2025). A comparative study of plantar pressure and inertial sensors for cross-country ski classification using deep learning. *Sensors*, *25*(5). https://doi.org/10.3390/s25051500

Prakash, S., Callahan, T., Bushagour, J., Banbury, C., Green, A. V., Warden, P., Ansell, T., & Reddi, V. J. (2023). CFU playground: Full-stack open-source framework for tiny machine learning (tinyml) acceleration on FPGAs. In *2023 IEEE International symposium on performance analysis of systems and software (ISPASS)* (pp. 157–167). https://doi.org/10.1109/ISPASS57527.2023.00024

Qi, L., Fan, J., Cai, H., & Fang, Z. (2024). A survey of emerging memory in a microcontroller unit. *Micromachines*, *15*(4). https://doi.org/10.3390/mi15040488

Qiu, S., Wang, Z., Zhao, H., Liu, L., Li, J., Jiang, Y., & Fortino, G. (2019). Body sensor network-based robust gait analysis: Toward clinical and at home use. *IEEE Sensors Journal*, *19*(19), 8393–8401. https://doi.org/10.1109/JSEN.2018.2860938

Raspberry pi pico w. (2025). https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html. Accessed: 2025-05-10.

Rafee, A. N. M., Zishan, M. A. O., & Noor, J. (2025). Decision tree-based real-time personalized human activity recognition and fall detection utilizing cost effective highly resource-constrained microcontrollers. https://doi.org/10.2139/ssrn.5146354

Stm32h747xi. (2025). https://www.st.com/en/microcontrollers-microprocessors/stm32h747xi.html. Accessed: 2025-05-10.

sedaghati, N., ardebili, S., & Ghaffari, A. (2025). Application of human activity/action recognition: A review. *Multimedia Tools and Applications*, *84*, 33475–33504. https://doi.org/10.1007/s11042-024-20576-2

Sonata, I., & Heryadi, Y. (2024). Comparison of LSTM and transformer for time series data forecasting. In *2024 7th International conference on informatics and computational sciences (ICICos)* (pp. 491–495). https://doi.org/10.1109/ICICoS62600.2024.10636892

Systems, E. (2023). Esp32-s3-devkitc-1 user guide. Accessed: 2024-09-29 https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/index.html.

Systems, E. (2025a). Current consumption measurement of modules. Accessed: 03-May-2024 https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-guides/current-consumption-measurement-modules.html.

Systems, E. (2025b). ESP-IDF sleep modes documentation. Accessed: 2025-05-28 https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/system/sleep_modes.html.

Systems, E. (2025c). Power management - ESP32-S3 - ESP-IDF programming guide. Accessed: 2025-03-12 https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/system/power_management.html.

Team, T. (2025). Callback. https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/Callback. TensorFlow Keras Callback API Documentation.

TensorFlow (2024). Tensorflow keras API documentation. Accessed: 2024-10-03 https://www.tensorflow.org/api_docs/python/tf/keras/ops.

TensorFlow, (2025). Adamw optimizer. Accessed: 2025-03-12 https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/AdamW.

Truong, P. H., You, S., Ji, S.-H., & Jeong, G.-M. (2021). Adaptive accumulation of plantar pressure for ambulatory activity recognition and pedestrian identification. *Sensors*, *21*(11). https://doi.org/10.3390/s21113842

Tsoukas, V., Gkogkidis, A., Boumpa, E., & Kakarountas, A. (2024). A review on the emerging technology of tinyML. *ACM Computing Surveys*, *56*(10), 259, 1–37. https://doi.org/10.1145/3661820

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023). Attention is all you need. https://arxiv.org/abs/1706.03762.

Wang, R., Gao, Z., Zhang, L., Yue, S., & Gao, Z. (2025a). Empowering large language models to edge intelligence: A survey of edge efficient LLMs and techniques. *Computer Science Review*, *57*, 100755. https://doi.org/https://doi.org/10.1016/j.cosrev.2025.100755

Wang, X., Tang, Z., Guo, J., Meng, T., Wang, C., Wang, T., & Jia, W. (2025b). Empowering edge intelligence: A comprehensive survey on-device AI models. *ACM Computing Surveys*, *57*(9), 1-39. https://doi.org/10.1145/3724420

Wit-Motion (2025). Bwt901cl: 9-axis bluetooth gyroscope accelerometer sensor. Accessed: 03-May-2025 https://www.wit-motion.com/BLE/50.html.

WITMOTION (2024). Witstandardprotocol JY901 - arduino SDK. Accessed: 26-May-2024 https://github.com/WITMOTION/WitStandardProtocol_JY901/tree/main/Arduino/Arduino_sdk.

Yang, C., Yu, X., An, Z., & Xu, Y. (2023). Categories of response-based, feature-based, and relation-based knowledge distillation. https://arxiv.org/abs/2306.10687.

Zeyer, A., Bahar, P., Irie, K., Schlüter, R., & Ney, H. (2019). A comparison of transformer and LSTM encoder decoder models for ASR. In *2019 IEEE Automatic speech recognition and understanding workshop (ASRU)* (pp. 8–15). https://doi.org/10.1109/ASRU46091.2019.9004025

Zhu, Y., Luo, B., Qiu, Q., & Zhu, T. (2024). Multimodal fusion contrastive learning framework based on insole and wristband. In *4th International conference on internet of things and smart city (ioTSC 2024)* (pp. 96–101). SPIE (*vol. 13224*). https://doi.org/10.1117/12.3034835